

Documentation

<https://bob.ibm.com/docs/ide>

Generated on: May 15, 2026

Table of Contents

1.0.2	4
General	7
System requirements	9
Understanding and using modes	14
Introduction	23
Tool workflow	43
Understanding the context window	47
Symptoms of context poisoning	54
Understanding context limits in large projects	58
Available keyboard commands	61
What is is.bobignore?	70
Why use custom modes	74
What are custom rules?	84
Overview	95
Components of the chat interface	98
To auto-approve actions:	108
Why use different modes?	118
Types of mentions	125
Why use Bob tips?	134
Overview	140
What are checkpoints	152
Why use code actions?	163
What is Next Edit?	169
Why use code reviews?	173
Why use this feature?	183
Why use this feature?	186
Why use this feature?	192
Why use literate coding?	195
Why use skills	198

Accessing Bobalytics	211
Overage	219
Ownership and administration	225
Security checklist	231
Enterprise features	238
Connection and authentication	243

1.0.2

April 2026

New Features

Bobalytics for Enterprise plans

Track Bob's impact on your software delivery lifecycle with comprehensive analytics and insights. Measure Bob's contribution to your codebase with the Bob Factor metric. Monitor Bobcoin spending across your organization. Analyze usage patterns by programming language and mode. View team activity and daily active users. Compare individual and team usage to optimize workflows.

[Learn more →](#)

API key management availability

Admin UI now includes API key management for supported administrative workflows. List all API keys with search and filtering. View API key details in a modal. Revoke API keys individually or in bulk. Show status badges for Active, Revoked, and Expired keys. Toggle visibility of revoked keys.

[Learn more →](#)

Bug Fixes

Code Predictions

Fixed an issue where code suggestions weren't updating when you changed code after the cursor. Bob now provides fresh, context-aware suggestions based on your current code state.

Init Command

Corrected typo in the init command: "rules-advance" is now "rules-advanced".

Task Tracking

Fixed how task IDs are assigned, making it easier to debug issues and correlate tasks across the system.

Literate Coding

Fixed an issue where extra line breaks were added before and after changes. Quick suggestions are now disabled in Literate Coding mode to prevent distractions while you write.

Installer

Fixed critical installation issues on Linux and Mac that prevented Bob from starting after installation. Corrected branding text in the Windows installer.

1.0.1

March 2026

New Features

AI-powered code generation

Turn natural language into working code across multiple programming languages. Generate new code from descriptions and requirements. Refactor and debug existing code with help from Bob. Get intelligent code completion with single-line and multi-line suggestions.

[Learn more →](#)

Specialized modes

Access purpose-built modes optimized for different development scenarios. Code mode writes, modifies, and refactors code. Ask mode provides answers and explanations about your codebase. Plan mode helps

design and plan implementations before coding. Advanced mode provides extended capabilities including MCP and browser tools. Orchestrator mode coordinates multi-step projects across specialties.

[Learn more →](#)

Comprehensive tool system

Accomplish complex development tasks efficiently. Read, write, and manipulate files directly within your project. Run terminal commands with intelligent error handling. Control and automate web browser actions for testing and validation. Search codebases and list code definitions for better context.

[Learn more →](#)

MCP (Model Context Protocol) integration

Extend IBM Bob's capabilities with custom tools and integrations. Connect to databases and APIs from within your editor. Access specialized development and operations tools. Integrate with your organization's internal systems.

[Learn more →](#)

Code review and git integration

Streamline your development workflow with built-in git support. Review code changes with the `/review` command. Generate commit messages automatically. Create pull requests with the `/create-pr` command. Track code attribution with git notes.

[Learn more →](#)

Custom modes and slash commands

Extend functionality for your team's needs. Define custom modes for specialized workflows. Create reusable command shortcuts with slash commands. Share mode configurations across your team. Apply persistent context with Bob Rules in `.bob/rules/` directories.

[Learn more →](#)

Skills system

Teach Bob reusable workflows with custom skills. Create instruction sets for code reviews, testing, documentation, and specialized tasks. Define skills in `.bob/skills/` directories with supporting files like checklists and templates. Share standardized workflows across your team through version control. Skills automatically activate in Advanced mode when relevant to your task.

[Learn more →](#)

General

Usage

Advanced Features

Troubleshooting

System requirements

Operating Systems

macOS, Linux, or Windows

Memory

Minimum 4 GB RAM (8 GB recommended)

Storage

At least 500 MB available disk space

Network

Active internet connection

[Download IBM Bob](#)

[Get the installer](#)

[Choose the correct version for you operating system.](#)

1. Download, then open the `.pkg` file for macOS.
 2. Complete the steps in the installation wizard.
-
1. Download the `.exe` installer for Windows.
 2. Run the downloaded `.exe` installer.
 3. Follow the installation wizard prompts.
 4. Choose your installation directory (default is recommended).
 5. Click **Finish** to complete the installation.

[Sign in with your IBMid](#)

Note:

An IBMid is required to authenticate. If you do not have one, see [Create an IBMid](#).

Open Bob from your applications menu or desktop shortcut. On first launch, Bob will prompt you to sign in:

1. Follow the authentication flow with your browser to login with your IBMid.
2. Return to Bob after authentication is complete.

Note:

If you experience network issues with outbound traffic, you might need to configure your firewall or proxy settings. For more information, see [Configuring firewall rules](#) or [Configuring proxy settings](#).

[Next steps](#)

Explore the following topics to get started using Bob:

Best practices

Learn effective strategies for working with Bob

Chat interface

Explore Bob's chat interface and features

Modes

Learn about specialized modes and when to use them

Custom modes

Create specialized workflows

Custom instructions

Add project-specific preferences

MCP servers

[Configure extended capabilities](#)

Security guidelines

[Understand best practices for secure usage](#)

Getting help

If you need assistance:

FAQ

[Find answers to common questions](#)

Understanding and using modes

Bob's modes are task-specific personas that shape its behavior. Selecting the appropriate mode helps you get the best results.

Built-in modes

Bob comes with several built-in modes, each designed for specific purposes:

Mode	Purpose	When to use
Code	Writing and modifying code	For implementing features, fixing bugs, or making code improvements
Advanced	Complex development tasks	When you need full access to all tools, including MCP and browser capabilities
Plan	Planning and designing	Before implementation, when you need to plan architecture or create technical specifications
Ask	Getting information	When you need explanations or information without modifying files
Orchestrator	Multi-step project coordination	For complex projects requiring coordination across different specialties

Mode selection strategy

For the most effective workflow:

Start with Plan mode for new projects or complex features to create a detailed implementation plan before writing any code. Read the plan carefully to confirm it aligns with your goal.

Use Orchestrator mode for complex, multi-step projects that require coordination across different specialties.

Switch to Code mode for most day-to-day development tasks when you are ready to implement.

Use Ask mode when you need explanations or information without modifying files.

Switch to Advanced mode when you need access to additional tools like browser automation or MCP servers.

For more details on modes and how to switch between them, see [Modes](#).

Custom modes

You can create specialized modes for specific tasks or workflows:

- Create modes for specialized tasks like security reviews or documentation writing.
- Configure custom modes with specific tool access permissions and role definitions.
- Share modes with your team to standardize workflows.

Learn more about creating and configuring custom modes in the [Custom modes](#) documentation.

Workflow optimization

Plan before coding

Always start complex projects or features in Plan mode to:

- Generate a detailed implementation plan.
- Break down complex problems into manageable steps.
- Identify potential challenges before they arise.
- Create a roadmap for implementation.

This approach prevents breaking changes and provides a clear direction for development.

Use checkpoints effectively

Checkpoints automatically version your workspace files during Bob tasks, enabling:

- Safe experimentation with AI-suggested changes.
- Easy recovery from undesired modifications.
- Comparison of different implementation approaches.

Tip

If Bob starts producing subpar work, use the "Restore Files & Task" checkpoint option to roll back to a previous state. This is more effective than trying to correct flawed output with new instructions.

To restore your work to a previous state:

Scroll up in your chat interface to locate a checkpoint you want to restore from.

Click the "Restore Checkpoint" button.

Click "Restore Files and Task".

Click "Confirm".

Monitor usage and optimize costs

Track your Bobcoin consumption and usage patterns with [Bobalytics](#) to optimize your workflow and manage costs effectively.

Integrate with version control

Regularly commit and push changes to your version control system:

- Create frequent, small commits with clear messages.
- Push changes to remote repositories regularly.
- Use branches for experimental features.

Git provides an essential safety net to prevent accidental, permanent deletion of code.

Communication strategies

Write effective prompts

The quality of your prompts directly affects the quality of Bob's responses:

- **Be specific and clear:** Vague prompts lead to vague outputs. Detail what Bob should and should not do.
- **Provide examples:** When possible, include examples of the desired output format or style.
- **Use the Enhance Prompt feature:** Click the enhance button to automatically improve your query with additional context and structure.

Learn more about the Enhance Prompt feature in the [Enhance prompt](#) documentation.

Use context mentions

Context mentions let you reference specific elements of your project directly in your conversations with Bob:

- Use `@/path/to/file.js` to include specific file contents.
- Use `@/path/to/folder` to include all files in a directory.
- Use `@problems` to include Bob Findings panel diagnostics.
- Use `@terminal` to include recent terminal output.
- Highlight text and use the keyboard shortcut `⌘ + L` to add it to the chat.

This approach is more efficient than copying and pasting code or describing file locations.

Learn more about context mentions in the [Context mentions](#) documentation.

Manage the context window

To prevent Bob from mixing up your goals:

- Start new tasks regularly with specific aims.
- Avoid giving Bob your entire codebase at once.
- Use direct file references to provide targeted context.
- Break complex tasks into smaller, focused subtasks.

Security and control

Configure auto-approval settings

Bob allows you to control which actions require your approval:

- **Manual approval:** Review and approve every action Bob takes (safest setting).
- **Auto-approve:** Grant Bob the ability to run specific tasks without interruption.
- **Hybrid approach:** Auto-approve low-risk actions while requiring confirmation for riskier tasks.

Configure these settings based on your comfort level and the sensitivity of your project.

Learn more about auto-approval settings in the [Auto-approving actions](#) documentation.

Use .bobignore

The `.bobignore` file lets you specify files and directories that Bob should not access or modify:

Create a `.bobignore` file in your project root.

Add patterns for sensitive files, build artifacts, and large assets.

Use the same syntax as `.gitignore`.

This helps protect sensitive information and prevent accidental changes to generated files.

Learn more about controlling file access in the [Using .bobignore](#) documentation.

For more detailed information on security guidelines, see [Security guidelines](#).

[Set up rules](#)

Bob allows you to specify rules to control how Bob performs tasks:

- Define rules globally or per project in the `.bob` directory.
- Create mode-specific rules in `.bob/rules-{mode-slug}/` directories.
- Use rules to enforce coding standards, documentation requirements, or other workflow constraints.

For more information on custom modes and rules, see the [Custom modes](#) documentation.

[Introduction](#)

IBM Bob is an AI SDLC (Software Development Lifecycle) partner that augments your existing workflows. In this tutorial, you start with an existing TypeScript Express REST API that manages to-do lists and use Bob to build a web UI on top of it. You do not need experience in Node.js, React, or TypeScript to complete this tutorial.

[Key features you learn](#)

You use the following three core capabilities of Bob to build the UI:

- **Modes:** Specialized personas that tailor Bob's behavior for specific tasks.
- **Approval workflow:** Review Bob plans before they run.
- **Iterating with Bob:** Have a conversation with Bob to refine work after you see the result.

[Prerequisites](#)

To complete this tutorial, you need the following:

IBM Bob IDE

Download and install the IBM Bob application on your computer. Bob is a standalone IDE application and not an extension.

Docker

Use Docker to run containerized builds. Docker prevents you from having to install Node.js or dependencies locally. If you do not have Docker, you might be able to use an alternative container management tool.

Git

Install Git to clone the example repository. Bob uses Git to clone repositories directly from the IDE.

Set up your workspace

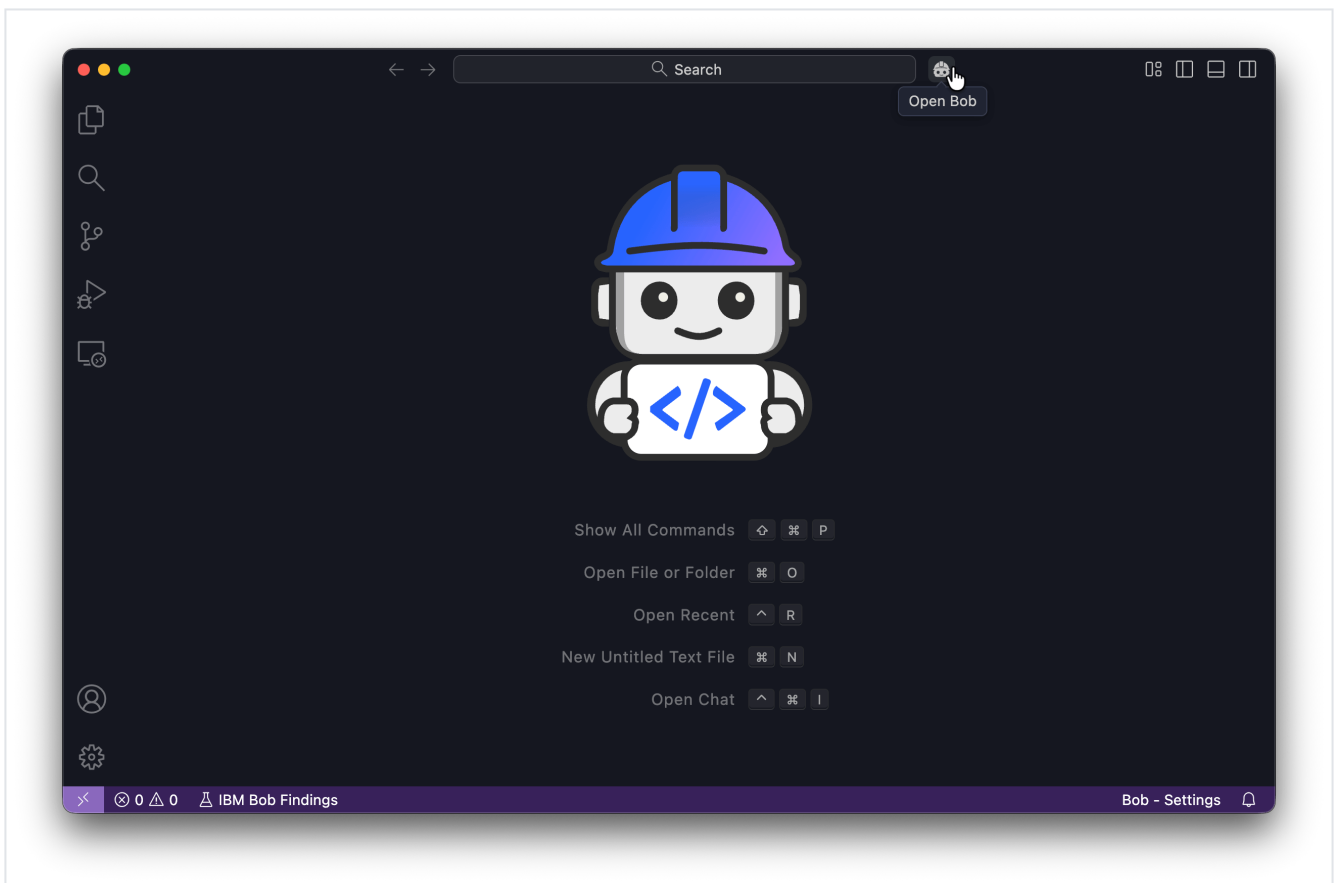
Launch IBM Bob and clone the example repository to begin working with the Node.js application.

Launch IBM Bob

Launch the IBM Bob application on your computer.

Open the Bob chat panel

If the chat panel is not already opened, open it by clicking the Bob icon beside the navigation bar or use the shortcut `Option + Command + B` (Mac) or `Ctrl + Alt + B` (Windows).



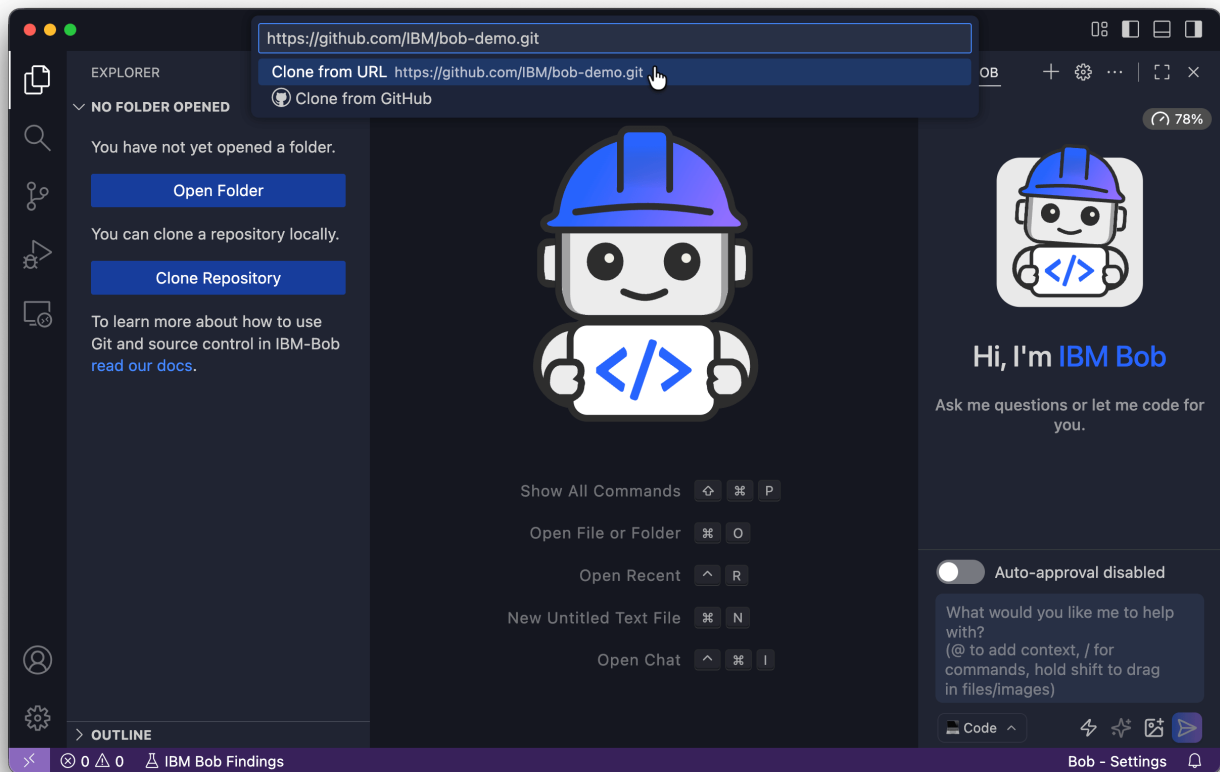
You interact with Bob by typing natural language requests in the input field.

[Clone the tutorial repository](#)

Clone the repository containing the example application using the following steps:

- Click the files icon in the top-left corner of the Bob IDE panel.
- Click **Clone Repository**, and paste the following GitHub repository into the search bar. Bob asks you where you want to save the repository. You can save it wherever you like.

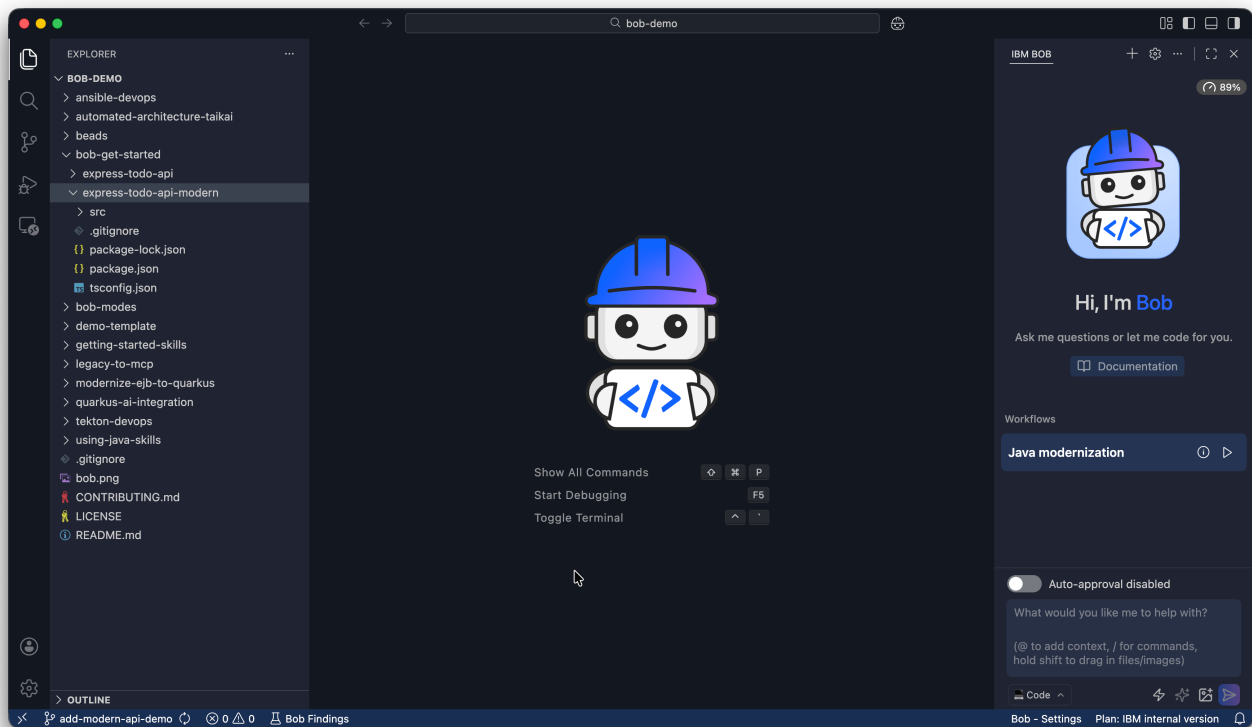
```
https://github.com/IBM/bob-demo.git
```



Open the repository

When Bob asks you if you want to open the cloned repository click **Open**. If Bob asks "Do you trust the authors of the files in the folder?", click **Yes, I trust the authors**.

The explorer shows the project files, and the Bob chat panel opens on the side. You work out of the **bob-get-started/express-todo-api-modern** folder.



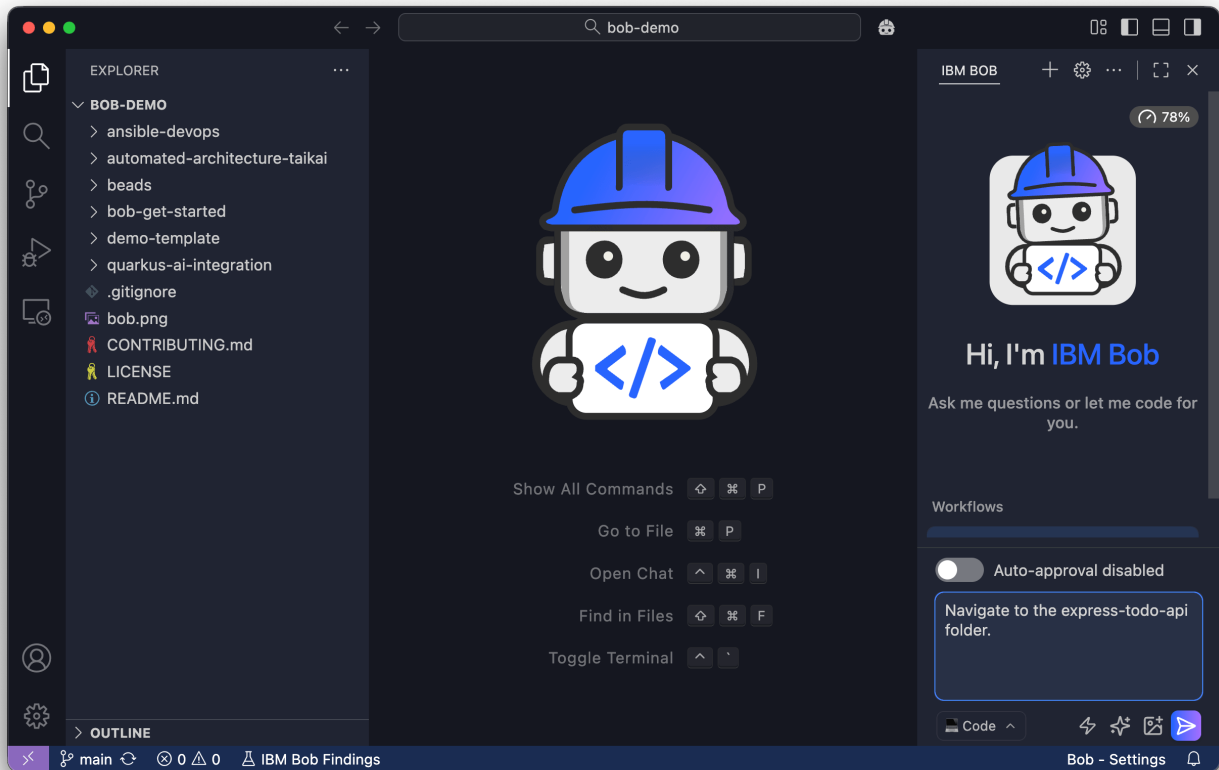
[Disable auto-approval](#)

Find the auto-approval toggle in the bottom-right corner of the window. If it's on, you see two small white checkmarks inside the toggle. Click the circle to turn it off. When it's off, you see **Auto-approval disabled**.

[Navigate to the project](#)

In the Bob chat panel at the bottom right of the window, enter the following prompt to navigate to the `express-todo-api-modern` folder. Bob asks you to let it run the change directory command. Click **Run** to let Bob change directories:

```
Navigate to the express-todo-api-modern folder.
```



Bob's responses might vary slightly from the examples in this tutorial, but its key actions should be the same. Bob confirms it navigated to the directory and describes the contents:

```
Successfully navigated to the express-todo-api-modern folder.
```

```
The directory contains a TypeScript Express.js TODO API project with the following structure:
```

- Source code in src/ (index.ts, db.ts, models/, routes/)
- Configuration files (package.json, tsconfig.json, .gitignore, .dockerignore)
- README.md documentation

Initialize Bob in your project

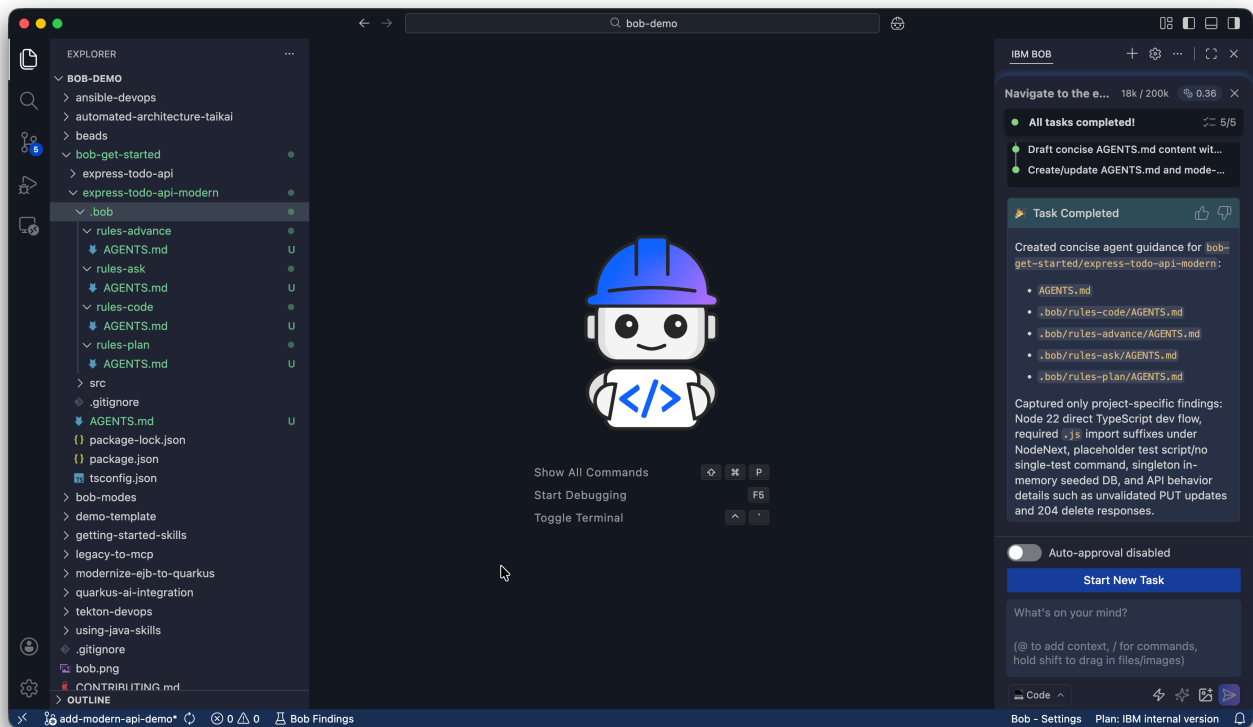
Run `/init` in the Bob chat panel to have Bob create a root `AGENTS.md` file and rule `AGENTS.md` files.

The root `AGENTS.md` file tell Bob the project's overall structure, conventions, and goals so Bob can write code that fits your project. Without it, Bob starts from scratch each time.

The rule files in `.bob` are where Bob creates `AGENTS.md` for each of its modes: advance, ask, code, and plan. Each mode has a different set of rules that guide Bob's behavior in that mode.

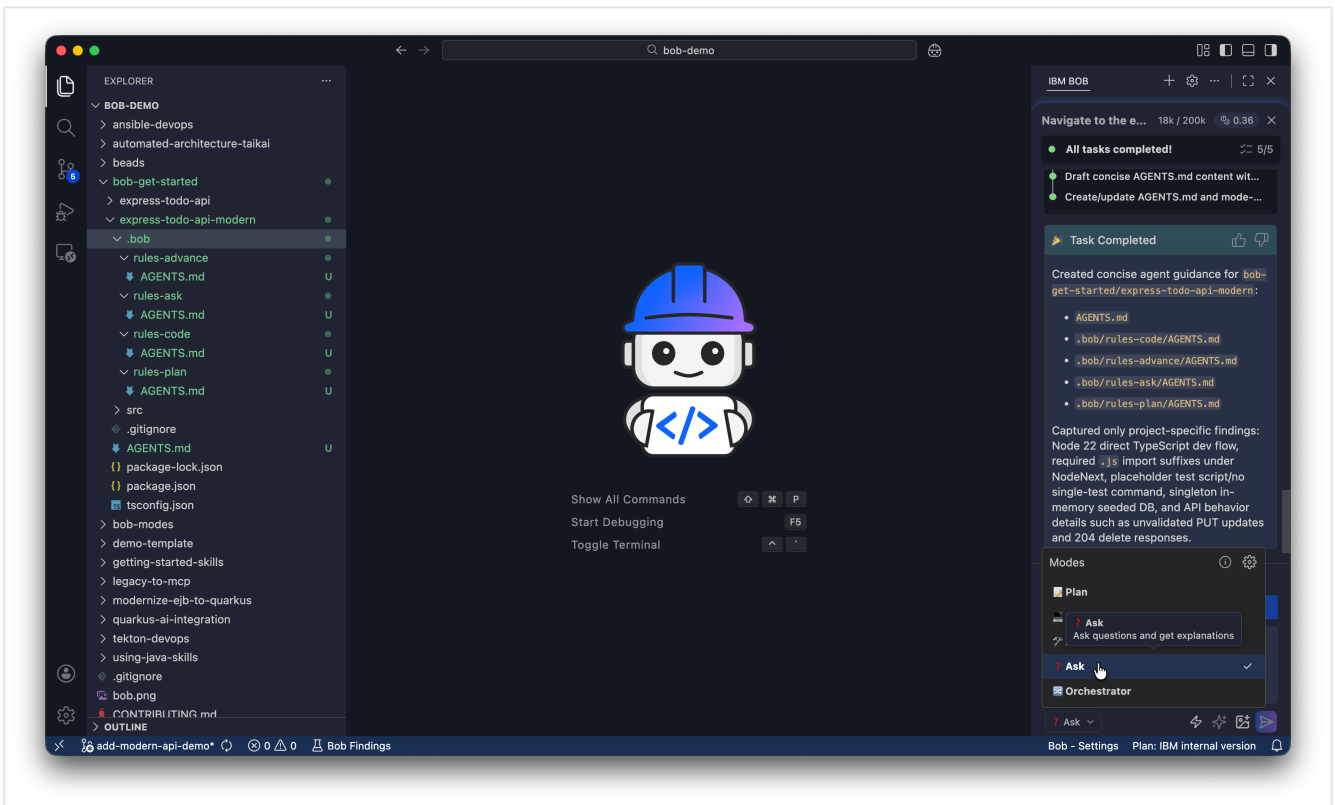
```
/init
```

Bob asks for permission to create the `AGENTS.md` files. Click **Approve** to create the files and **Save** to save them. Once Bob creates all of the `AGENTS.md` files, you see the following files:



[Explore the API with Ask mode](#)

Before changing anything, switch to Bob's **Ask mode**. Click the mode selector drop-down to the left of the chat input field and select **Ask**, or type `/ask` in the chat.



Ask mode can only read files, not write them. Bob can't change anything in this mode, so it's safe to explore. [Modes](#) are one way Bob respects the principle of least privilege.

Ask Bob to explain the project:

```
Explain what this application does and how it works.
```

Bob reads through the project and explains what it does.

Add a task to the to-do list

Switch to Code mode

Click the mode selector drop-down and select **Code**, or type `/code` in the chat. Code mode lets Bob write, modify, and refactor code. Bob still asks for permission before each operation.

Build and run the API in Docker

Ask Bob to create a Dockerfile and start the container:

```
Create a Dockerfile for this API using Node 22 and run it on port `3000`.
Don't install dependencies locally.
The Dockerfile handles all installations.

Once you create the Dockerfile, build the image and start the container.
```

Bob asks you to approve building the image and starting the container. Click **Approve** and **Save** for each.

Bob might find an NPM issue, such as the following one. Approve the fixes Bob proposes:

```
The build failed because there's no package-lock.json file.
I need to update the Dockerfile to use npm install instead of npm ci:
```

The API is now running and available.

[Add a task to the to-do list](#)

To test the API, paste the following command directly into the Bob chat panel. This command adds a task to the to-do list by making a POST request to the API:

```
curl -X POST http://localhost:3000/api/todos \  
  -H "Content-Type: application/json" \  
  -d '{"title": "Build a web UI"}'
```

Bob shows you the response, which confirms you can add tasks to the to-do list with the API.

```
{  
  "id": "a1b2c3d4-e5f6-...",  
  "title": "Build a web UI",  
  "completed": false,  
  "createdAt": "2026-02-16T16:15:56.627Z"  
}
```

[Stop and remove the container](#)

Ask Bob to stop and remove the container. You build a new container with the web UI in the following section:

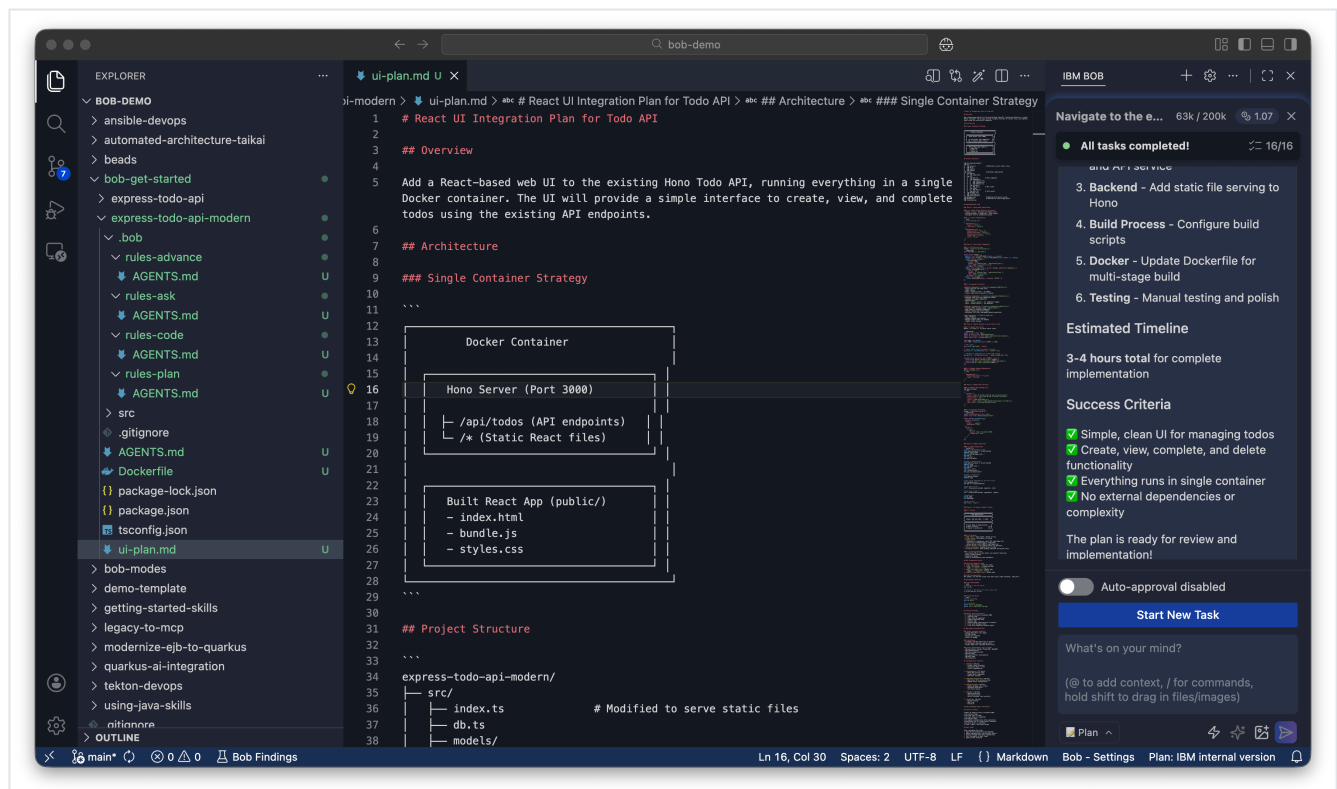
```
Stop and remove the running container and Docker image.
```

Plan the web UI with Bob

Before modifying code, switch to **Plan mode** so Bob can create a plan on how it builds the web UI. Click the mode selector dropdown and select **Plan**, or type `/plan` in the chat.

Ask Bob to plan the web UI as `ui-plan.md` in the `express-todo-api-modern` project directory:

```
Plan how you would add a React web UI to this todo app.  
I want to be able to submit new tasks, see a list of existing tasks, and check  
tasks as completed.  
Use the existing API endpoints to manage tasks, and keep the UI simple.  
Everything will run from a single Docker container.  
  
Save the plan to a file named `ui-plan.md` in the express-todo-api-modern project  
directory.
```



Review the plan before telling Bob to run it. Click `ui-plan.md` in the explorer to open it.

Build a web UI with Bob

Ask Bob to build the web UI

Switch Bob to **Code mode** by clicking the mode selector drop-down and selecting **Code**, or type `/code` in the chat.

Ask Bob to build the UI, containerize it, and run it:

```
Implement the plan `ui-plan.md` you outlined to build the web UI
containerize the application with Docker, and run it.
Don't install dependencies locally. The Dockerfile handles all installations.
```

Bob reads the `ui-plan.md`, shows you the files it will create or change, and then builds and starts the container. Click **Approve**, **Run**, and **Save** for each file Bob creates or modifies.

Fix build failures

Sometimes Bob's first attempt at building the UI and container might fail due to errors in the code or Dockerfile. Bob shows you the error message and proposes a fix. Development iteration is part of writing code with Bob.

The following are some possible errors and fixes you might encounter:

```
The build failed because `npm ci` requires a package-lock.json file.  
Let me update the Dockerfile to use `npm install` instead.
```

```
Vite can't find index.html. The issue is that index.html is in the `public`  
directory  
but Vite expects it in the root. Let me move it to the correct location.
```

[Open the web UI in your browser](#)

Once the container is running, Bob gives you a summary similar to the following:

Successfully implemented the UI plan and containerized the Todo application! 🎉

What was accomplished:

Backend (Hono + Node.js 22)

- Created `src/models/todo.ts` - TypeScript interfaces
- Created `src/db.ts` - In-memory database with sample data
- Created `src/routes/todos.ts` - RESTful API routes (GET, POST, PUT, DELETE)
- Created `src/index.ts` - Hono server with static file serving
- Created `package.json` - Dependencies and build scripts
- Created `tsconfig.json` - TypeScript configuration with ESM support

Frontend (React + TypeScript + Vite)

- All client files were already created (App, components, services, types)
- Vite configured with proxy for API calls during development

Docker

- Multi-stage Dockerfile builds both client and server
- Production image optimized with only necessary files
- Successfully built image: `todo-app`
- Container running on port 3000

Application is now running:

- **UI**: <http://localhost:3000>
- **API**: <http://localhost:3000/api/todos>

Verified functionality:

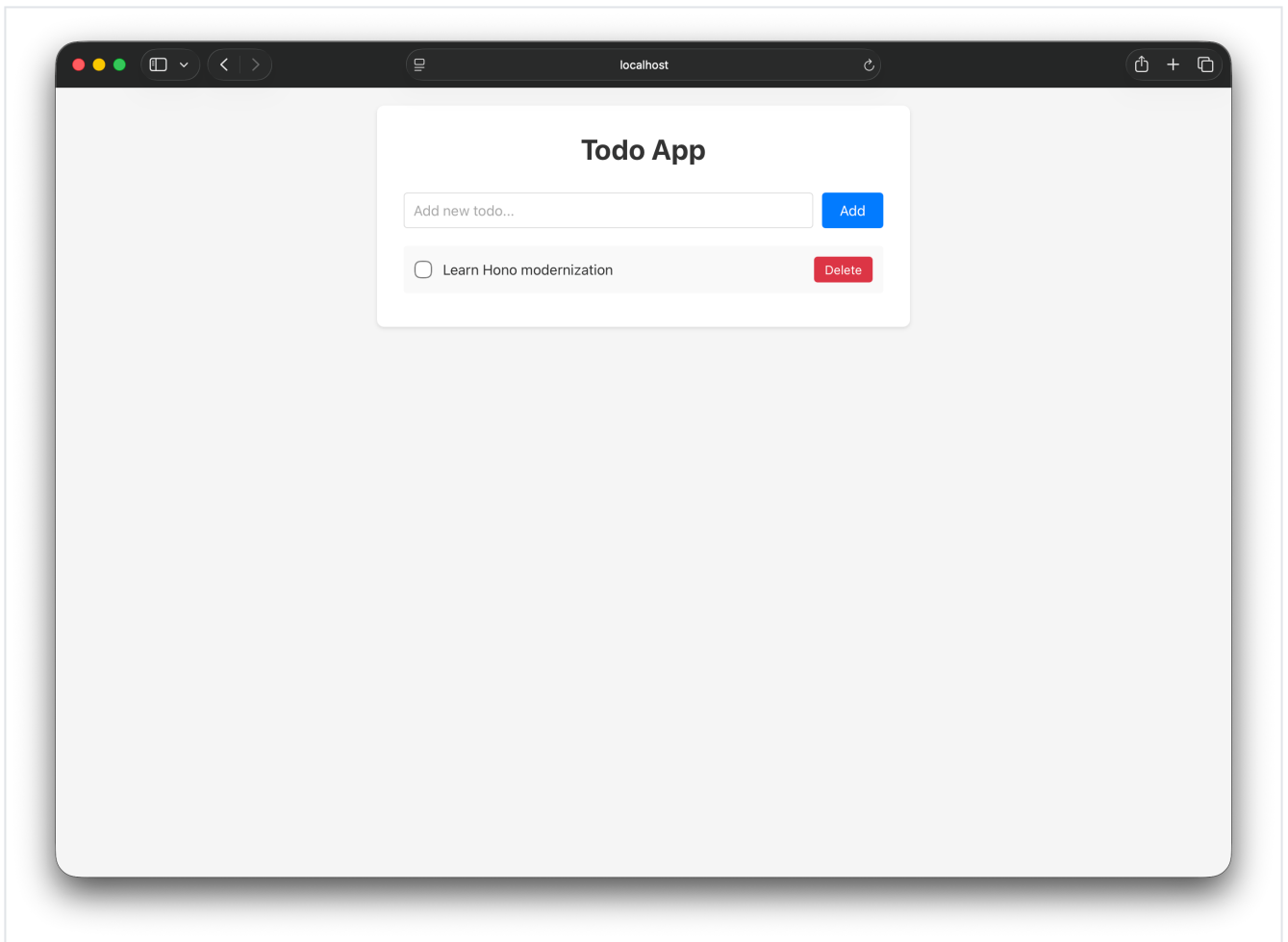
- ☒ API returns sample todo: "Learn Hono modernization"
- ☒ Created new todo via POST request
- ☒ UI HTML is being served correctly
- ☒ Single container serves both API and React UI

To manage the container:

- View logs: `docker logs todo-app-container`
- Stop: `docker stop todo-app-container`
- Restart: `docker start todo-app-container`
- Remove: `docker rm -f todo-app-container`

The application follows the plan exactly - modern Hono backend with React frontend, all running in a single optimized Docker container!

Open <http://localhost:3000> in your browser. You see the "Build a web UI" task that you added previously. Try adding a new task using the input field to confirm the API is working correctly.



Customize the UI

Ask Bob to update the styles

Personalize the UI by telling Bob what font, font color, and background color you want. The following are some fonts you can choose from:

- **Inter:** A popular choice for web apps
- **Georgia:** A classic serif font, which is warm and readable
- **JetBrains Mono:** A monospace font with a developer-tool feel

Pick a font color and background color. In the Bob chat panel, describe your choices using natural language. The following is a customization example:

```
Update the styles of the web UI.  
Use the Inter font.  
Dark navy background.  
White text.
```

Bob proposes edits to the React component styles. Review the diff and click **Save** to accept the changes.

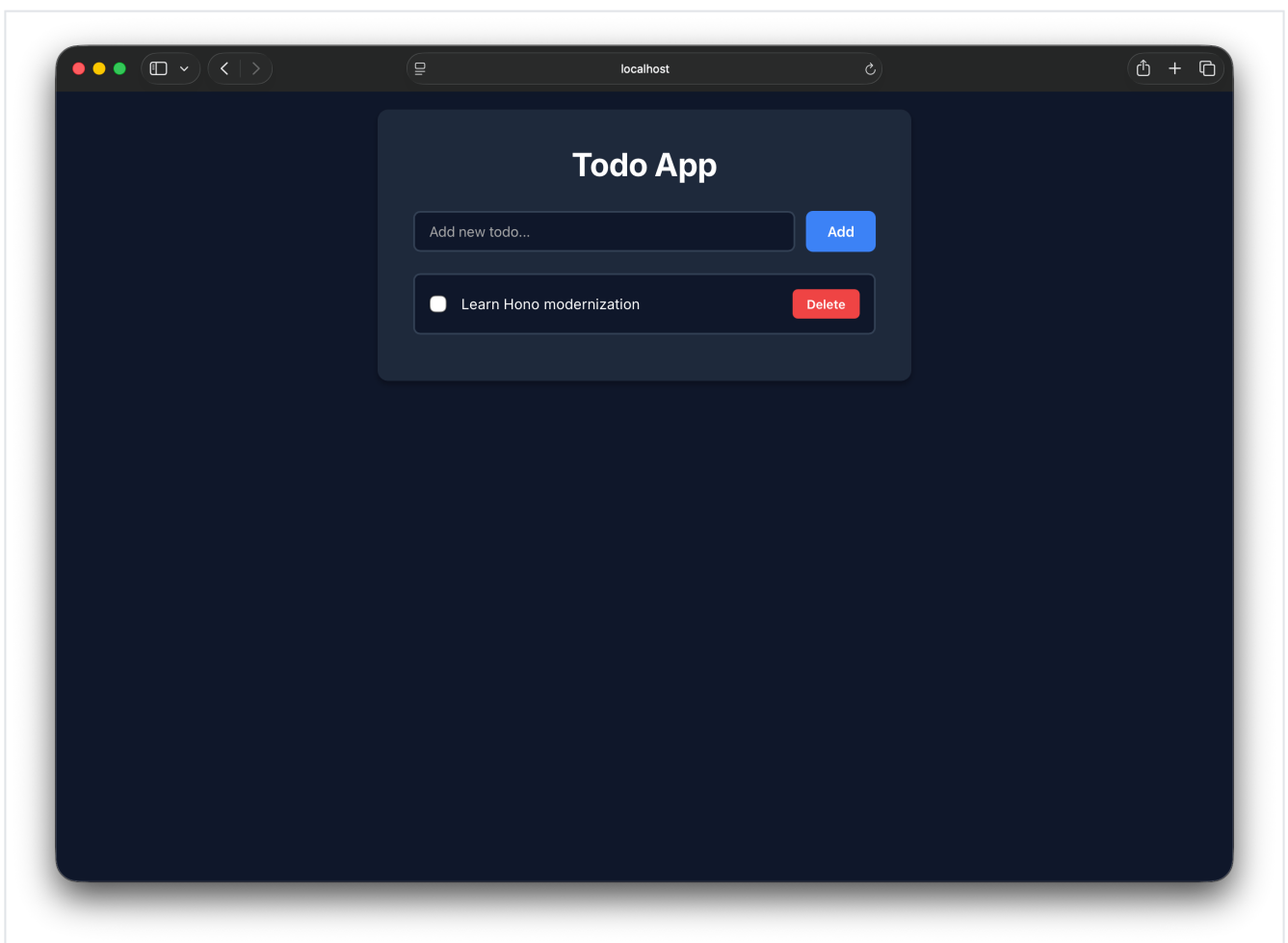
Rebuild the container and view your changes

The running container still serves the old styles. Ask Bob to rebuild and restart it so you can see your changes:

```
Rebuild the Docker image and restart the container with the updated styles.
```


Refresh `http://localhost:3000` in your browser to see the updated design.

Because this API application is stateless, when you rebuild the Docker container, the to-do list resets to the default tasks. In a real application with a database, your data would persist through container rebuilds.



Clean up

When you're done, ask Bob to stop and remove the container:

```
Stop and remove the running container and the Docker image.
```

Next steps

In this tutorial, you used Bob to build a web UI for an existing Express API and ran the result in Docker. You used Ask mode to safely explore the project, Plan mode to review Bob's approach, Code mode to build and customize the UI, and the approval workflow to stay in control of every change. Continue building on what you learned with the following resources:

- Read the [Bob security guidance](#) to learn how to safely operate Bob and protect your code and credentials.
- Read [Bob best practices](#) to learn effective prompting, mode selection, and workflow strategies for using Bob.
- Explore the [Bob get started tutorials](#) to learn more about Bob's capabilities.
- Follow the [modernize a Node.js Express API](#) tutorial to practice using Bob on a real refactoring task.

Tool workflow

When you work with Bob, it determines the appropriate actions. For each step, Bob will:

1. Select the appropriate tool based on your request
2. Present the tool with its parameters for your review
3. Run the approved tool and show you the results
4. Continue this process until your task is complete

Tool categories

Bob's tools are grouped by their primary function. Understanding these categories helps you see how Bob performs different tasks.

Read tools

Read tools let Bob access file content and understand code structure without making changes.

Tool	Purpose	Example use
<code>read_file</code>	Read the contents of one or more files	View configuration files, examine source code
<code>search_files</code>	Search across files using regex	Find TODOs, locate function definitions
<code>list_files</code>	List files and directories	Explore project structure, find file types
<code>list_code_definition_names</code>	Extract class, function, and method names	Get an overview of code organization

Bob uses read tools when your request involves reviewing code, searching for patterns, or examining project structure.

[Write tools](#)

Write tools let Bob create new files or modify existing code.

Tool	Purpose	Example use
<code>write_to_file</code>	Create a new file or overwrite an existing file	Generate new components, create configuration files
<code>apply_diff</code>	Apply targeted changes to specific parts of a file	Update function logic, fix bugs, refactor code
<code>insert_content</code>	Insert content at a specific position in a file	Add imports, insert new functions

Bob uses write tools when your request involves creating files, implementing features, fixing bugs, or refactoring code.

Command tools

Command tools let Bob run commands and perform system operations.

Tool	Purpose	Example use
<code>execute_command</code>	Run CLI commands in your workspace	Install dependencies, run tests, build projects

Bob uses command tools when your request involves running commands, installing packages, running scripts, or performing system operations.

MCP tools

MCP tools let Bob extend its capabilities through connected Model Context Protocol (MCP) servers.

Tool	Purpose	Example use
<code>use_mcp_tool</code>	Access tools provided by connected MCP servers	Query databases, interact with APIs, access external services

Bob uses MCP tools when your request involves capabilities provided by configured MCP servers.

Mode tools

Mode tools let Bob switch between different modes for specialized tasks.

Tool	Purpose	Example use
<code>switch_mode</code>	Change to a different mode (Code, Plan, Ask, etc.)	Switch to Plan mode for architecture design, Code mode for implementation

Bob uses mode tools when your request is better handled by a different mode and its specialized capabilities.

Question tools

Question tools let Bob gather additional information needed to complete tasks.

Tool	Purpose	Example use
<code>ask_followup_question</code>	Request clarification or additional detail	Ask about preferred implementation approach, request missing information

Bob uses question tools when your request requires more information or clarification before Bob can proceed.

Understanding the context window

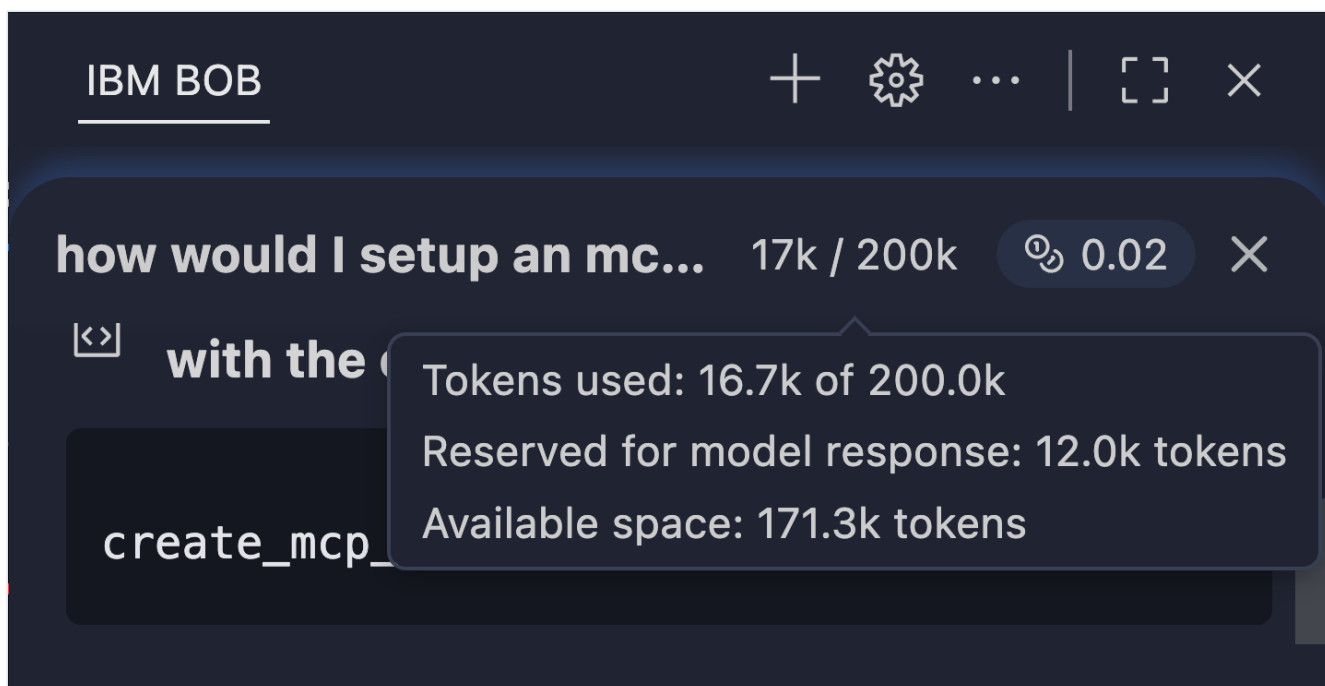
The context window is the total amount of information Bob can process at once, measured in tokens.

The context window includes:

- **System instructions:** Bob's core capabilities, tool definitions, and mode-specific instructions.
- **Conversation history:** All messages, tool uses, and results from your current chat.
- **File contents:** Any files you reference using @ mentions.
- **Tool outputs:** Results from commands, file reads, searches, and other operations.
- **MCP tool definitions:** Descriptions and parameters for all connected MCP server tools.

Monitoring your token usage

Bob displays the current token usage in the top-right corner of the chat panel. This counter shows how many tokens are being used in the current conversation, helping you track context consumption in real time.



[Token limitation](#)

Bob's context window has a maximum capacity of **200,000 tokens**. When you approach this limit, Bob automatically condenses the conversation at **140,000 tokens** to maintain performance and prevent context overflow.

[Automatic context condensation](#)

When a conversation reaches **140,000 tokens**, Bob automatically:

1. Preserves the most recent and relevant context.
2. Summarizes or removes older conversation segments.
3. Maintains critical system instructions and tool definitions.
4. Continues the conversation with condensed context.

Bob continues operating with earlier conversation context summarized.

Impact on Bobcoins

Context window usage affects your Bobcoin consumption. Every token processed by Bob, both input and output, contributes to your usage:

- **Larger context processes more tokens:** Each interaction includes the full active context.
- **Repeated processing:** The same context gets processed with every message you send.
- **Cumulative effect:** Long conversations with extensive context consume more Bobcoins per interaction.

Best practices for context management

Keep conversations focused

- Start a new chat when switching to a different task or topic. This prevents unrelated context from consuming tokens and potentially confusing Bob.
- Use one chat for implementing a new feature, a separate chat for fixing an unrelated bug, and another chat for refactoring a different component.
- Avoid using the same chat for multiple unrelated tasks, continuing conversations indefinitely across different projects, or mixing debugging, feature development, and documentation in one chat.

Use context mentions strategically

Context mentions are useful but consume tokens. Be selective about what you include.

Consider the following examples:

Efficient mention usage:

```
@/src/components/Button.tsx Refactor this component to use TypeScript
```

Inefficient mention usage:

```
@/src @/tests @/docs Review everything and suggest improvements
```

The second example includes unnecessary content, increasing token and Bobcoin usage.

Provide targeted context

Instead of mentioning entire directories or large files, focus on specific sections:

Targeted approach:

```
@/src/utils/validation.ts:45-67 Fix the email validation logic
```

Broad approach:

```
@/src/utils Review all utility functions
```

Using a targeted approach gives Bob exactly what it needs without consuming unnecessary tokens.

You can also highlight text in your editor and use `CMD + L` (Mac) or `Ctrl + L` (Windows/Linux) to add it to the chat.

Consider when to use MCP tools

Each MCP server tool you connect adds its definition to Bob's context window. While MCP tools extend Bob's capabilities, having numerous tools can consume significant tokens.

- Only connect MCP servers you actively use.
- Disconnect unused MCP servers to free up context space.
- Consider using project-specific MCP configurations rather than global ones.

You can enable and disable specific tools for an MCP server, to ensure you are only using the tools you need.

Working with large projects

Large projects require additional context management strategies to work effectively within token limits. When working with codebases that have many files or complex architectures:

- Break down large tasks into smaller, focused subtasks.
- Use specific file paths and function names rather than vague references.
- Target specific sections of large files using line ranges (e.g., `@/src/utils/validation.ts:45-67`).

- Make iterative changes, reviewing and approving each step before proceeding.

For detailed strategies and examples, see [Working with large projects](#).

[Learn more](#)

Learn more about [managing Bob's context window](#) by starting a new conversation.

Context poisoning occurs when inaccurate or irrelevant data contaminates the language model's active context. This causes Bob to draw incorrect conclusions, provide erroneous information to tools, and progressively deviate from the intended task with each interaction.

Once a chat session's context is compromised, the only reliable solution is to start a new session. (Learn more about [managing Bob's context window](#).) Starting fresh with a clean context is crucial for maintaining accuracy and effectiveness.

Symptoms of context poisoning

You can identify context poisoning by observing these behaviors:

- **Degraded output quality:** Suggestions become nonsensical, repetitive, or irrelevant.
- **Tool misalignment:** Tool calls no longer correspond to your requests.
- **Orchestration failures:** Orchestrator mode chains may stall, loop indefinitely, or fail to complete.
- **Temporary fixes:** Reapplying a clean prompt or instructions offers only brief relief before issues resurface.
- **Tool usage confusion:** Bob struggles to correctly use or recall how to use tools defined in the system prompt.

Common causes

Context poisoning can be triggered by several factors:

- **Model hallucination:** Bob generates incorrect information and subsequently treats it as factual context.
- **Code comments:** Outdated, incorrect, or ambiguous comments in the codebase can be misinterpreted, leading Bob down the wrong path.
- **Contaminated user input:** Copy-pasting logs or text containing hidden or rogue control characters.
- **Context window overflow:** As a session grows, older useful information may be pushed out of the context window, allowing poisoned data to have greater relative impact.

Once bad data enters the context, it tends to persist. Bob reevaluates this tainted information in subsequent reasoning cycles, similar to a permanent flaw affecting perception until the context is completely reset.

Can a "wake-up prompt" resolve context poisoning?

Short answer: No.

A corrective prompt might temporarily suppress symptoms, but the problematic data remains in the conversational buffer. The model will likely revert to the poisoned state as soon as the interaction deviates from the narrow scope of the corrective prompt.

Detailed explanation:

- Reinjecting the full set of tool definitions or core directives can sometimes mask the damage for one or more interactions following the initial context poisoning.
- However, the underlying poisoned context remains. Any query or task outside the immediate patch will likely retrigger the original issue.
- This approach is unreliable, similar to placing a warning label on a leaking pipe instead of repairing it.

Effective recovery strategies

To reliably recover from context poisoning:

- **Start a new session:** The most dependable solution is to start a new chat session. This clears the contaminated context entirely.
- **Be selective with data:** When pasting logs or other data, only include the essential information Bob requires.
- **Break down complex tasks:** For large or complex tasks, break them into smaller, focused chat sessions. This helps ensure that stale or irrelevant information ages out of the context window more quickly.
- **Validate tool output:** If a tool returns nonsensical or clearly incorrect data, delete that message from the chat history before Bob can process it and incorporate it into context.

Addressing a common question: The "magic bullet" prompt

A frequent question from the community is:

"Have you found a prompt that wakes it back up? Maybe a prompt that just has the tools instructions we can push back in manually?"

No single prompt offers a lasting fix. Any immediate improvement is superficial because the corrupted text persists in the session's history, ready to cause further issues. The only robust solution is to discard the compromised session and start a new one with a clean prompt.

Understanding context limits in large projects

Bob uses large language models that have a limited context window. This is the maximum amount of text (measured in tokens) that Bob can process at once. If the context is too large, Bob may not be able to understand your request or generate accurate responses.

The [context window](#) includes:

- The system prompt (instructions for Bob)
- The conversation history
- The content of any files you mention using @
- Selected text sent to chat using `Cmd + L` (Mac) or `Ctrl + L` (Windows/Linux)
- The output of any commands or tools Bob uses

Strategies for managing context

1. **Be specific:** When referring to files or code, use specific file paths and function names. Avoid vague references like "the main file."

2. **Use context mentions effectively:** Use `@/path/to/file.ts` to include specific files. Use `@problems` to include current errors and warnings. Use `@` followed by a commit hash to reference specific Git commits. You can also highlight text in your editor and use `Cmd + L` (Mac) or `Ctrl + L` (Windows/Linux) to send it directly to chat.
3. **Break down tasks:** Divide large tasks into smaller, more manageable subtasks. This helps keep the context focused.
4. **Summarize when needed:** If you need to refer to a large amount of code, consider summarizing the relevant parts in your prompt instead of including the entire code.
5. **Prioritize recent history:** Bob automatically truncates older messages in the conversation history to stay within the context window. Be mindful of this and reinclude important context if needed.
6. **Configure large file handling:** Adjust the large file handling setting to control how Bob processes files that exceed a certain size threshold. This helps manage context usage when working with large files.
7. **Manage MCP tool usage:** You can disable and enable specific tools in an MCP service to allow you to use only what you need, reducing context consumption from unused tool definitions.

Example: Refactoring a large file

If you need to refactor a large TypeScript file (`src/components/MyComponent.tsx`), here is an effective approach:

1. Get an initial overview:

```
@/src/components/MyComponent.tsx List the functions and classes in this file.
```

2. Target specific functions:

```
@/src/components/MyComponent.tsx Refactor the `processData` function to use  
`async/await` instead of Promises.
```

3. **Make iterative changes:** Make small, incremental changes, reviewing and approving each step.

By breaking down the task and providing specific context, you can work effectively with large files within the context window limits.

Available keyboard commands

Bob offers keyboard commands to enhance your workflow. This page focuses on the `Bob.acceptInput` command, but here is a quick reference to key keyboard commands:

Command	Description	Default Shortcut
<code>Bob.acceptInput</code>	Submit text or accept the primary suggestion	None (configurable)
<code>Bob.focus</code>	Focus the Bob input box	None (configurable)
Inline edit	Edit code directly in the editor	Cmd + K (Mac) / Ctrl + K (Windows/Linux)
Move to chat	Send selected code to the chat interface	Cmd + L (Mac) / Ctrl + L (Windows/Linux)
Literate coding mode	Toggle literate coding mode	Cmd + I (Mac) / Ctrl + I (Windows/Linux)

For a complete list of all available keyboard shortcuts and to configure custom bindings, open **Bob** Settings, under **General** settings and navigate to the **Keyboard shortcuts** option where you can view and modify all shortcuts.

Key benefits of keyboard commands

- **Keyboard-driven interface:** Submit text or select the primary suggestion button without mouse interaction
- **Improved accessibility:** Essential for users with mobility limitations or those who experience discomfort with mouse usage
- **Vim/Neovim compatibility:** Supports seamless transitions for developers coming from keyboard-centric environments
- **Workflow efficiency:** Reduces context switching between keyboard and mouse during development tasks

Bob.acceptInput command

The `Bob.acceptInput` command lets you submit text or accept suggestions with keyboard shortcuts instead of clicking buttons or pressing Enter in the input area.

How the command works

The `Bob.acceptInput` command is a general-purpose input submission command. When triggered, it:

- Submits your current text or image input when in the text input area (equivalent to pressing Enter)
- Clicks the primary (first) button when action buttons are visible (such as confirm/cancel buttons or any other action buttons)

Detailed setup guide

Method 1: Using the UI

1. Open the Command Palette (`Ctrl+Shift+P` or `Cmd+Shift+P` on Mac)
2. Type "Preferences: Open Keyboard Shortcuts"
3. In the search box, type "Bob.acceptInput"
4. Locate "Bob: Accept Input/Suggestion" in the results
5. Click the + icon to the left of the command
6. Press your desired key combination (e.g., `Ctrl+Enter` or `Alt+Enter`)
7. Press Enter to confirm

Method 2: Editing keybindings.json directly

1. Open the Command Palette (Ctrl+Shift+P or Cmd+Shift+P on Mac)
2. Type "Preferences: Open Keyboard Shortcuts (JSON)"
3. Add the following entry to the JSON array:

```
{
  "key": "ctrl+enter", // or your preferred key combination
  "command": "Bob.acceptInput",
  "when": "BobViewFocused" // This is a context condition that ensures the
command only works when Bob is focused
}
```

You can also use a more specific condition:

```
{
  "key": "ctrl+enter",
  "command": "Bob.acceptInput",
  "when": "webviewViewFocus && webviewViewId == 'Bob-cline.SidebarProvider'"
}
```

Recommended key combinations

Choose a key combination that does not conflict with existing IBM Bob shortcuts:

- Alt+Enter - Easy to press while typing

- `Ctrl+Space` - Familiar for those who use autocomplete
- `Ctrl+Enter` - Intuitive for running commands
- `Alt+A` - Mnemonic for "Accept"

Practical use cases

Quick development workflows

- **Text submission:** Send messages to Bob without moving your hands from the keyboard
- **Action confirmations:** Accept operations like saving files, running commands, or applying diffs
- **Multi-step processes:** Move quickly through steps that require confirmation or input
- **Consecutive tasks:** Chain multiple tasks together with minimal interruption

Keyboard-centric development

- **Vim/Neovim workflows:** If you are coming from a Vim/Neovim background, maintain your keyboard-focused workflow
- **IDE integration:** Use alongside other IBM Bob keyboard shortcuts for a seamless experience

- **Code reviews:** Quickly accept suggestions when reviewing code with Bob
- **Documentation writing:** Submit text and accept formatting suggestions when generating documentation

Accessibility use cases

- **Hand mobility limitations:** Essential for users who have difficulty using a mouse
- **Repetitive strain prevention:** Reduce mouse usage to prevent or manage repetitive strain injuries
- **Screen reader integration:** Works well with screen readers for visually impaired users
- **Voice control compatibility:** Can be triggered via voice commands when using voice control software

Accessibility benefits

The `Bob.acceptInput` command was designed with accessibility in mind:

- **Reduced mouse dependence:** Complete entire workflows without reaching for the mouse
- **Reduced physical strain:** Helps users who experience discomfort or pain from mouse usage
- **Alternative input method:** Supports users with mobility impairments who rely on keyboard navigation

- **Workflow optimization:** Particularly valuable for users coming from keyboard-centric environments like Vim/Neovim

Keyboard-centric workflows

Here are some complete workflow examples showing how to effectively use keyboard shortcuts with Bob:

Development workflow example

1. Open IBM Bob and navigate to your project
2. Open Bob via the sidebar
3. Type your request: "Create a REST API endpoint for user registration"
4. When Bob asks for framework preferences, use your `Bob.acceptInput` shortcut to select the first suggestion
5. Continue using the shortcut to accept code generation suggestions
6. When Bob offers to save the file, use the shortcut again to confirm
7. Use IBM Bob's built-in shortcuts to navigate through the created files

[Code review workflow](#)

1. Select code you want to review and use IBM Bob's "Copy" command
2. Ask Bob to review it: "Review this code for security issues"
3. As Bob asks clarifying questions about the code context, use your shortcut to accept suggestions
4. When Bob provides improvement recommendations, use the shortcut again to accept implementation suggestions

[Troubleshooting](#)

Issue	Solution
Shortcut does not work	Ensure Bob is focused (click in the Bob panel first)
Wrong suggestion selected	The command always selects the first (primary) button; use mouse if you need a different option
Conflicts with existing shortcuts	Try a different key combination in IBM Bob keyboard settings
No visual feedback when used	This is normal - the command silently activates the function without visual confirmation
Shortcut works inconsistently	Make sure the <code>when</code> clause is properly configured in your <code>keybindings.json</code> (either <code>BobViewFocused</code> or the webview-specific condition)

Technical implementation

The `Bob.acceptInput` command is implemented as follows:

- Command registered as `Bob.acceptInput` with display title "Bob: Accept Input/Suggestion" in the command palette
- When triggered, it sends an "acceptInput" message to the active Bob webview
- The webview determines the appropriate action based on the current UI state:
 - Clicks the primary action button if action buttons are visible and enabled
 - Sends the message if the text area is enabled and contains text/images
- No default key binding - users assign their preferred shortcut

Limitations

- Works only when the Bob interface is active
- Has no effect if no inputs or suggestions are currently available
- Prioritizes the primary (first) button when multiple options are shown

What is .bobignore?

- **Purpose:** To protect sensitive information, prevent accidental changes to build artifacts or large assets, and generally define Bob's operational scope within your workspace.
- **How to use:** Create a file named `.bobignore` in the directory of your workspace. List patterns in this file to tell Bob which files and directories to ignore.

Bob actively monitors the `.bobignore` file. Any changes you make are reloaded automatically, ensuring Bob always uses the most current rules. The `.bobignore` file itself is always implicitly ignored, so Bob cannot change its own access rules.

Pattern syntax

The syntax for `.bobignore` is identical to `.gitignore`. Here are common examples:

Example	Description
<code>node_modules/</code>	Ignores the entire <code>node_modules</code> directory
<code>*.log</code>	Ignores all files ending in <code>.log</code>
<code>config/secrets.json</code>	Ignores a specific file
<code>!important.log</code>	An exception; Bob will <i>not</i> ignore this specific file, even if a broader pattern like <code>*.log</code> exists

Example	Description
<code>build/</code>	Ignores the <code>build</code> directory
<code>docs/**/*.md</code>	Ignores all Markdown files in the <code>docs</code> directory and its subdirectories

To learn about syntax, refer to the [official Git documentation on .gitignore](#).

How Bob tools interact with .bobignore

`.bobignore` rules are enforced across various Bob tools:

Strict enforcement (reads & writes)

These tools directly check `.bobignore` before any file operation. If a file is ignored, the operation is blocked:

- `read_file`: Will not read ignored files.
 - `write_to_file`: Will not write to or create new ignored files.
 - `apply_diff`: Will not apply diffs to ignored files.
-

- `list_code_definition_names`: Will not parse ignored files for code symbols.

File editing tools (potential write bypass)

The `insert_content` and `search_and_replace` tools use an internal component for managing changes. **Important:** Currently, the final write operation performed by these tools might bypass `.bobignore` rules. While initial read attempts might be blocked, the save action itself does not have an explicit check.

File discovery and listing

- **list_files tool:** When Bob lists files, ignored files are typically omitted or marked with a  symbol (see "User Experience" below).
- **Environment details:** Information about your workspace (like open tabs and project structure) provided to Bob is filtered to exclude or mark ignored items.

Command execution

- **execute_command tool:** This tool checks if a command (from a predefined list like `cat` or `grep`) targets an ignored file. If so, running the command is blocked.

Key limitations and scope

- **Workspace-centric:** `.bobignore` rules apply **only to files and directories within the current Bob workspace root**. Files outside this scope are not affected.
- **execute_command specificity:** Protection for `execute_command` is limited to a predefined list of file-reading commands. Custom scripts or uncommon utilities might not be caught.
- **Write operations via insert_content & search_and_replace:** As noted, these tools might be able to write to ignored files due to current limitations in their save mechanism.
- **Not a full sandbox:** `.bobignore` is a powerful tool for controlling Bob's file access via its tools, but it does not create a system-level sandbox.

User experience and notifications

- **Visual cue (🔒):** In file listings, files ignored by `.bobignore` may be marked with a lock symbol (🔒), depending on the `showbobignoredFiles` setting (defaults to `true`).
- **Error messages:** If a tool operation is blocked, Bob receives an error: "Access to [file_path] is blocked by the `.bobignore` file settings. You must try to continue in the task without using this file, or ask the user to update the `.bobignore` file."
- **Chat notifications:** You will typically see a notification in the Bob chat interface when an action is blocked due to `.bobignore`.

This guide helps you understand the `.bobignore` feature, its capabilities, and its current limitations, so you can effectively manage Bob's interaction with your codebase.

Why use custom modes

- **Specialization:** Optimize modes for specific tasks like documentation writing, test engineering, or security reviews
- **Safety:** Restrict commands or file access for sensitive operations
- **Team collaboration:** Share standardized workflows across your team
- **Experimentation:** Test different configurations without affecting other modes

Mode components

Component	Description
Slug	Unique identifier used internally and for mode-specific instruction files
Name	Display name shown in the Bob interface
Role definition	Core identity and expertise that defines Bob's personality and behavior
When to use	(Optional) Guidance for when to use this mode, used by Orchestrator for task coordination
Available tools	Tool groups and file access permissions the mode can use
Custom rules	(Optional) Additional behavioral guidelines or rules

How to create custom modes

You can create custom modes by asking Bob with the "Mode writer" mode, using the settings menu, or manually editing configuration files.

Use the "Mode writer" mode

1.

Click the  icon in the Bob panel to open Settings.

2. Select the **Modes** tab.

3.

Click the  button to create a new mode.

[Use the settings menu](#)

1.

Click the  icon in the Bob panel to open Settings.

2. Select the **Modes** tab.

3.

Click the  button to create a new mode.

4. Fill in the fields for **Name**, **Slug**, **Save Location**, **Role Definition**, **When to Use (optional)**, **Available Tools**, and **Custom Instructions**.

5. Click **Save**.

Bob saves the new mode in YAML format. You can add file type restrictions for the `edit` tool group by asking Bob or through manual YAML configuration.

[Edit configuration files manually](#)

You can manually edit mode configuration files in YAML format:

- **Global modes:** Edit `custom_modes.yaml` via Settings → Modes → Edit Global Modes
- **Project modes:** Edit `.bob/custom_modes.yaml` in your project. Click Settings → Modes → Edit Project Modes

These files define an array of custom modes in YAML format.

Example (`custom_modes.yaml` or `.bob/custom_modes.yaml`):

```
customModes:
  - slug: docs-writer
    name: 📄 Documentation Writer
    roleDefinition: You are a technical writer specializing in clear
documentation.
    whenToUse: Use this mode for writing and editing documentation.
    customInstructions: Focus on clarity and completeness in documentation.
    groups:
      - read
      - - edit
        - fileRegex: \.(md|mdx)$
          description: Markdown files only
      - browser
```

Mode configuration properties

Available tool groups

- `read`: Read files and directories
- `edit`: Modify files (can be restricted with `fileRegex`)
- `browser`: Use browser automation
- `command`: Execute terminal commands
- `mcp`: Access MCP servers
- `skill`: Access skills

File restrictions for the edit tool

Restrict which files a mode can edit using `fileRegex` in YAML format (single backslash):

```
groups:
  - read
  - - edit
  - fileRegex: \.(js|ts)$
    description: JavaScript and TypeScript files only
```

Common regex patterns

Pattern	Matches	Example files
\.md\$	Markdown files	readme.md, docs/guide.md
^src/.*	Files in src directory	src/app.js, src/components/button.tsx
\.(css scss)\$	CSS and SCSS files	styles.css, theme.scss
^(?!.*(test spec)).*\.(js ts)\$	JS/TS files excluding tests	app.js, utils.ts (not app.test.js)

Tip:

Ask Bob to generate regex patterns for you instead of writing them manually.

Add mode-specific instructions

You can add mode-specific instructions using either a directory structure (preferred) or a single file.

Preferred method (directory structure):

01-style-guide.md

02-formatting.txt

Alternative method (single file):

```
.bobrules-{mode-slug}
```

The directory method takes precedence if both exist. Files in the directory are loaded alphabetically and combined with the `customInstructions` property from your mode configuration.

To add mode-specific instructions:

1. Create a `.bob/rules-{mode-slug}/` directory in your project root (or a `.bobrules-{mode-slug}` file).
2. Add instruction files to the directory (for example, `01-style-guide.md` and `02-formatting.txt`).

Bob automatically loads these instructions when you use the mode.

How to override default modes

You can override default Bob modes (such as Code, Ask, or Plan) by creating a custom mode with the same slug in your project configuration.

To override a default mode:

- 1.

Click the  icon in the Bob panel to open Settings.

2. Select the **Modes** tab.

3. Click **Edit Project Modes** to edit the `.bob/custom_modes.yaml` file.

4. Add your mode configuration with the slug of the default mode you want to override:

```
customModes:
  - slug: code # Matches default Code mode
    name: 🖥️ Code (Python Only)
    roleDefinition: You are a Python software engineer.
    whenToUse: Use for Python development tasks.
    customInstructions: Follow PEP 8 and use type hints.
    groups:
      - read
      - - edit
      - fileRegex: \.py$
        description: Python files only
      - command
```

Project-specific overrides take precedence over global overrides, which take precedence over defaults.

YAML format

YAML is the preferred and recommended format for custom modes:

YAML advantages:

- More readable with indentation-based structure
- Supports comments (#)
- Cleaner multi-line string syntax
- Less punctuation required
- Easier to edit and maintain

Automatic migration:

- Global modes: Automatically migrated from legacy `custom_modes.json` to `custom_modes.yaml` on startup
- Project modes: Converted to YAML when edited through the UI

While legacy JSON format files are still supported for backward compatibility, all new modes should be created in YAML format.

[Learn more](#)

Get hands-on and [extend Bob's capabilities by creating a custom product-manager mode](#).

What are custom rules?

Custom rules extend Bob's default behavior by defining preferences, constraints, and guidelines that direct how Bob approaches tasks to match your needs.

Common uses include:

- Coding style preferences (indentation, naming conventions)
- Documentation formats and standards
- Testing methodologies and requirements
- Project workflows and processes
- Team-specific conventions

Adding custom rules for a mode

You can add custom rules to modes in two ways:

[Through the UI](#)

Configure rules that apply across all your projects:

1. Click the three dots in the top-right corner of the Bob sidebar.
2. Click **Modes**.
3. Select the mode you want to edit rules for.
4. Click **Edit**.
5. Scroll to **Mode-specific custom instructions (optional)**.
6. Type your rules.
7. Click **Save**.

[Through mode-specific rules files](#)

Modes can also have additional rules files stored in directories:

- **Preferred method** (directory structure):

01-style-guide.md

02-formatting.txt

- **Alternative method** (single file):

```
.bobrules-{mode-slug}
```

The directory method takes precedence if both exist. Files in the directory are loaded alphabetically and combined with the `customInstructions` property from your mode configuration. For more details, see [Custom modes](#).

Learn more about [standardizing Bob's behavior using project-level rules](#).

Rule scopes

Bob supports two rule scopes that determine where your rules apply:

- **Global rules:** Apply automatically across all your projects
-

- **Workspace rules:** Apply only within your current project

Use global rules for personal or organization-wide standards. Use workspace rules for project-specific requirements.

Configuration methods

File-based configuration

The simplest approach uses single files in your workspace root:

- `.bobrules` - General workspace rules
- `.bobrules-code` - Rules for Code mode
- `.bobrules-{modeSlug}` - Rules for any mode

Create a `.bobrules` file:

```
# In your project root
echo "Use 4 spaces for indentation" > .bobrules
```

Directory-based configuration

For better organization, use directories:

Workspace structure:

coding-style.md

typescript.md

- `rules/` - General rules for all modes
- `rules-code/` - Code mode specific rules

Global structure:

Linux/macOS: `~/.bob/rules/` Windows: `%USERPROFILE%\bob\rules\`

Create workspace rules:


```
mkdir -p .bob/rules
echo "# Project standards" > .bob/rules/coding-style.md
```

Create global rules:

```
# Linux/macOS
mkdir -p ~/.bob/rules
echo "# Global standards" > ~/.bob/rules/coding-standards.md

# Windows
mkdir %USERPROFILE%\bob\rules
echo # Global standards > %USERPROFILE%\bob\rules\coding-standards.md
```

Rule priority

Bob combines rules from multiple sources in this order:

1. **Global rules** (~/.bob/rules/)
2. **Workspace rules** (.bob/rules/)

Within each level, mode-specific rules load before general rules. Workspace rules can override global rules.

Writing effective rules

Be specific and actionable

Good: "Use 4 spaces for indentation in JavaScript files"

Avoid: "Format code nicely"

Use clear structure

Organize rules by topic:

```
# Code Style
- Use camelCase for variables
- Use PascalCase for classes

# Testing
- Write unit tests for all public functions
- Use Jest for testing framework

# Documentation
- Add JSDoc comments for public APIs
```

Examples of effective rules

- "Always use spaces for indentation, with a width of 4 spaces"
- "Use camelCase for variable names"
- "Write unit tests for all new functions"
- "Explain your reasoning before providing code"
- "When adding features to websites, ensure they are responsive and accessible"

Advanced configuration

Mode-specific rules

Target specific modes with dedicated directories:

Directory	Purpose
rules/	General rules for all modes
rules-code/	Code mode only

Directory	Purpose
rules-plan/	Plan mode only
rules-{mode}/	Any custom mode

[Using AGENTS .md files](#)

For team standardization, you can use an AGENTS .md file in your workspace root:

```
# Team Standards
- Follow company coding guidelines
- Use approved libraries only
- Document all API changes
```

Key details:

- Automatically loaded by default
- Version-control with your project
- Disable with "bob-code.useAgentRules": false in settings
- Loaded after mode-specific rules but before general workspace rules

[File behavior](#)

- **Recursive reading:** Bob reads all files in rules directories, including subdirectories

- **Alphabetical order:** Files process in alphabetical order by filename
- **Automatic filtering:** Excludes cache files (.DS_Store, *.bak, *.cache, *.log, *.tmp, Thumbs.db)
- **Symbolic links:** Supported with maximum depth of 5
- **Empty files:** Silently skipped

Team standardization

Project-level standards

Use workspace .bob/rules/ directories under version control:

```
# Add to version control
git add .bob/rules/
git commit -m "Add Bob custom rules"
```

This ensures consistent behavior across team members for specific projects.

Organization-wide standards

Distribute global rules to team members:

1. Create a shared rules repository
2. Team members clone to `~/ .bob/rules/`
3. Update periodically for consistency

Hybrid approach

You can combine both approaches:

- Global rules for organization standards
- Workspace rules for project-specific requirements
- Workspace rules override global rules when needed

IBM Bob collects telemetry data by default to help improve the product. You can opt out to disable telemetry data collection at any time. Telemetry data helps IBM learn how to improve Bob. Your cooperation is greatly appreciated.

Note:

Enterprise plan users must keep telemetry enabled to view data in Bobalytics. Disabling telemetry will prevent your usage data from appearing in analytics dashboards.

Overview

When telemetry is enabled, IBM Bob collects usage data and telemetry about how you interact with the product. This includes:

- Actions you take in IBM Bob
- Features you use and how they perform
- Error and diagnostic information

This data helps IBM:

- Identify which features are most useful to users
 - Prioritize future development efforts
-

- Diagnose and fix issues
- Improve overall product quality

Privacy

IBM Bob does not collect your code, prompts, or other sensitive information as part of telemetry data.

Configuring telemetry data collection

You can control telemetry data collection through IBM Bob's settings.

1. In the top menu bar, click **IBM Bob > Settings > Settings**.
2. In the settings panel, navigate to **Application > Telemetry**.
3. Use the **Telemetry Level** dropdown to control data collection:
 - Select **off** to disable telemetry data collection
 - Select **all** to enable telemetry data collection (default)

1. In the top menu bar, click **File > Settings > Settings**.
 2. In the settings panel, navigate to **Application > Telemetry**.
 3. Use the **Telemetry Level** dropdown to control data collection:
 - Select **off** to disable telemetry data collection
 - Select **all** to enable telemetry data collection (default)
-

Telemetry Level setting

The **Telemetry Level** setting controls all telemetry data collection. Set it to **off** to completely disable data collection, or **all** to enable it.

Components of the chat interface

Chat history

Displays your conversation with Bob, including your requests, Bob's responses, and actions taken such as file edits or command executions.

Input field

Where you type your tasks and questions for Bob using natural language or use slash commands for quick actions.

Action buttons

Appear above the input field, allowing you to approve or reject Bob's proposed actions. Available buttons change depending on context.

Auto-approve toolbar

Located above the input field, allows you to enable auto-approval for specific actions like read, write, execute, and more to speed up your workflow.

Slash commands

Type '/' in the input field to access built-in commands (like /review, /init) and custom commands for common workflows.

Enhance prompt button

Located in the bottom right of the input field, automatically improves your prompt by adding clarity, structure, and context before sending.

Send button

Located to the right of the input field, sends your message to Bob.

New chat button

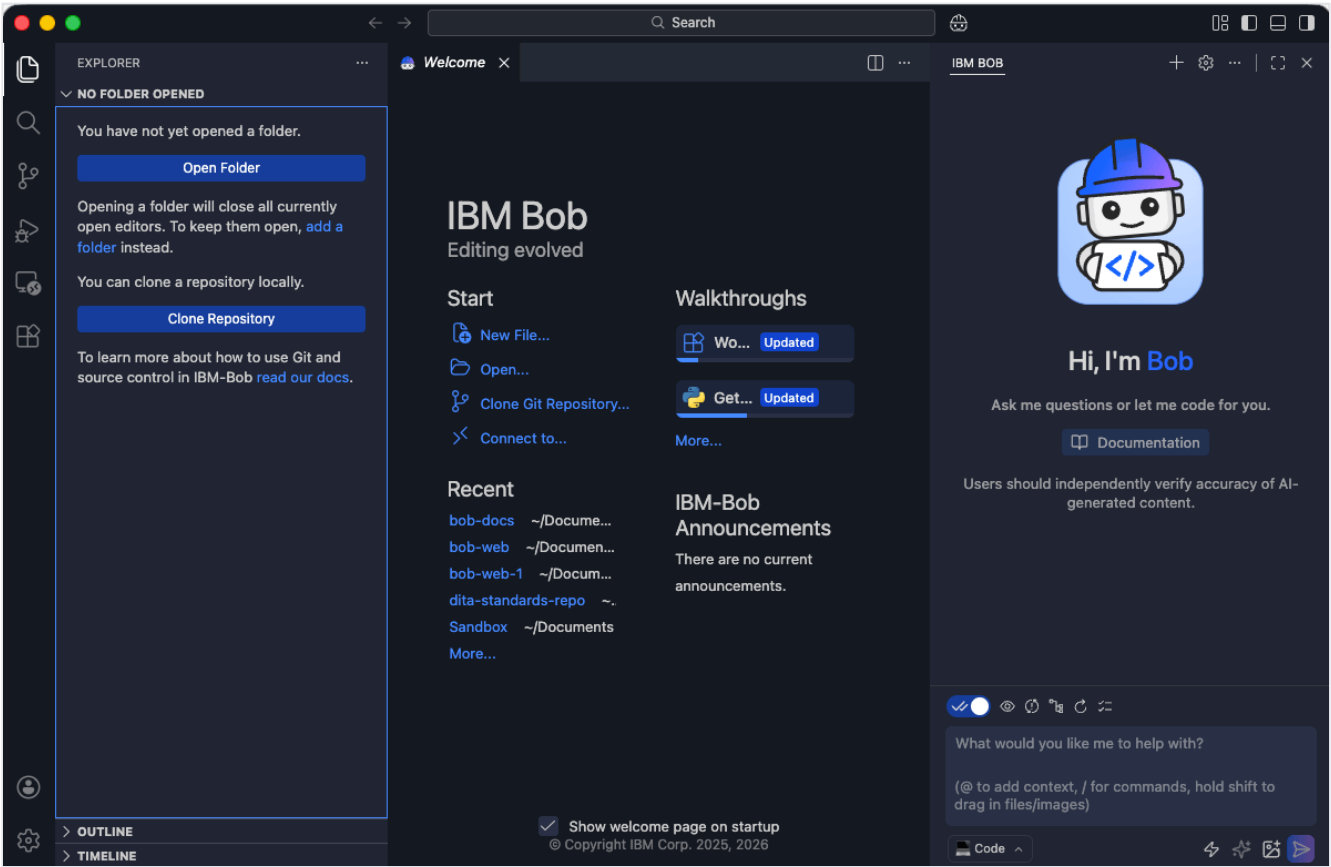
Located in the header, starts a new conversation session.

Settings button

Located in the header, opens settings to customize Bob's features and behavior.

Mode selector

A dropdown menu to the left of the chat input field for selecting which mode Bob should use for your tasks.



Writing effective prompts

Clear and specific requests yield the best results:

Strategy	Implementation
Be specific	"Fix the bug in <code>calculateTotal</code> that returns incorrect results when given negative numbers" instead of "Fix the code"

Strategy	Implementation
Provide context	Use @ context mentions to reference files and code sections
Break down tasks	Submit complex tasks as smaller, manageable steps
Include examples	Provide sample code when you need specific formatting or style

Example requests

Create a new file

Create a new file named ``utils.py`` and add a function called ``add`` that takes two numbers as arguments and returns their sum

Modify existing code

In the file `@src/components/Button.tsx`, change the color of the button to blue

Find and replace

Find all instances of the variable ``oldValue`` in `@/src/App.js` and replace them with ``newValue``

Explain code

Explain the function ``calculateTotal`` in `@/src/utils.ts`

Fix problems

`@problems` address all detected problems

Common pitfalls to avoid

Avoid	Use
Vague instructions such as "Make it better"	Clear, specific guidance such as "Improve error handling by adding try/catch blocks."
Relying on Bob to infer context	Explicit file and function references using @ mentions
Combining multiple unrelated tasks in a single request	One focused request at a time
Skipping validation of results	Reviewing Bob's output to ensure it meets your requirements

Tips for complex requests

For more complex tasks:

1. Start with a clear overview of what you want to accomplish
2. Provide relevant background information
3. Break down the task into logical steps
4. Use @ mentions to reference specific files or code sections
5. Review Bob's responses and provide feedback to refine results

Using the enhance prompt feature

The enhance prompt button () refines your request before sending by adding clarity, structure, and relevant context. This is particularly useful when:

- Your initial prompt is vague or needs more structure
- You want to ensure Bob understands your intent
- You need help breaking down a complex request

Example transformation:

Original prompt:

```
make a function to calculate tax
```

Enhanced prompt:

```
Create a JavaScript function that calculates sales tax. The function should:  
1. Take two parameters: price (number) and taxRate (decimal)  
2. Return the calculated tax amount  
3. Include proper error handling for invalid inputs  
4. Include comments explaining the calculation logic  
  
Provide the complete function implementation with an example of how to use it.
```

Bob's tip: Always review the enhanced prompt before sending to ensure it matches your intent.

Learn more about [enhancing prompts](#).

Interacting with messages

- **Clickable references:** File paths, URLs, and other references in the chat history are clickable. Clicking a file path opens the file in the editor, while clicking a URL opens it in your default browser.
- **Copying text:** Select text in the chat history and use the standard copy command (Ctrl + C / ⌘ + C). Code blocks include a dedicated "Copy" button for convenience.
- **Expanding and collapsing:** Click on a message to expand or collapse it, helping you manage longer conversations.

Status indicators

Indicator	Description
Loading spinner	Appears when Bob is processing your request.
Error messages	Displayed in red when an error occurs.
Success messages	Displayed in green when actions complete successfully.

Keyboard shortcuts

You can use keyboard shortcuts to interact with the chat interface more efficiently:

- `Enter`: Send your message (when input field is focused)
- `Shift + Enter`: Add a new line in your message

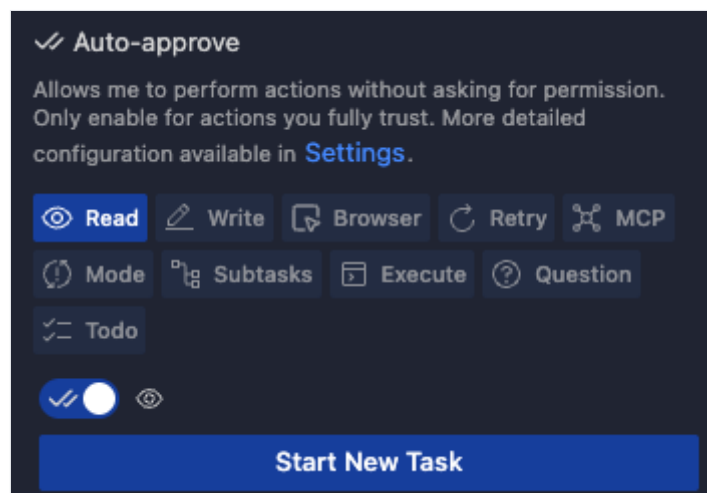
For more keyboard shortcuts and configuration options, see [Keyboard shortcuts](#).

Warning:

Auto-approve settings bypass confirmation prompts, giving Bob direct access to your system. This can result in **data loss, file corruption, or worse**. Command line access is particularly dangerous, as it can potentially run harmful operations that could damage your system or compromise security. Only enable auto-approval for actions you fully trust.

To auto-approve actions:

1. Hover over the Auto-Approve toolbar above the chat input.
2. Select which actions Bob can complete without asking for permission.



Available actions

When you auto-approve an action, Bob will complete the respective action without asking for permission.

Action	Description	Risk level
Read	View your files and directory content	Medium
Write	Create, edit, and save files to your directory	High
Browser	Gather information from websites using an internet browser	High
Retry	Attempt to connect to failed API requests	Low
MCP	Use MCP servers you have configured	Medium-High
Mode	Switch modes to complete a task	Low
Subtasks	Create and complete subtasks	Low
Execute	Run commands in your terminal	High
Question	After the time limit expires, select the first answer from the provided options.	Medium
Todo	Update the todo list	Low
Skills	Activate Skills you have defined	Medium

When to use auto-approve

Auto-approve is most beneficial in these scenarios:

- **Repetitive development tasks:** When you trust Bob's actions, such as generating basic code.
- **Batch operations:** When you need to process multiple files without interruption.
- **Exploratory coding sessions:** When you want to maintain flow without constant prompts.
- **Local development environments:** Where security risks are contained.

Understanding actions

Each action controls a specific workflow that Bob can complete.

Read

Bob accesses and reads files in your project without asking for permission.

This includes viewing file contents, listing directory structures, and searching through your codebase. This permission involves information gathering rather than making changes to your system.

Write

Bob creates, modifies, and deletes files without asking for permission.

Bob can apply code changes, refactor existing code, and manage file structure automatically. This directly modifies your codebase and involves high risk.

Browser

Bob can interact with websites through a headless browser without asking for permission.

Bob can navigate pages, click elements, fill forms, and capture screenshots to help with information gathering and web development tasks. **Web automation can perform unexpected actions**, including submitting forms, making purchases, or modifying data on websites. This action involves high risk.

Retry

Bob can automatically retry failed API requests without asking for permission.

When API calls fail due to temporary network issues or other reasons, Bob will attempt to reconnect automatically. This permission involves network operations but does not affect your local system or files.

MCP

Bob can use configured Model Context Protocol (MCP) servers without asking permission.

MCP servers provide additional tools and resources that extend Bob's capabilities. The risk level depends on what the specific MCP servers can access and modify.

Mode

Bob can change between different modes (Code, Plan, Ask, Advanced, Orchestrator) automatically based on task requirements.

This action involves workflow organization and does not affect system access.

Subtasks

Subtasks are separate task instances that Bob creates to break down complex work into manageable pieces. Bob can create and complete these subtasks automatically without asking for confirmation.

This permission relates to workflow organization rather than system access.

Execute

Bob can run terminal commands automatically without asking permission. This includes running build scripts, package managers, and other command-line tools.

Commands can complete system-level operations, and possibly run harmful processes. This action involves high risk.

Two-stage approval process

Execute auto-approval uses a two-stage security system:

1. **Top-level approval:** Enable the Execute toggle in the auto-approve toolbar to allow command execution.
2. **Command-specific approval:** Individual commands must be added to an allowed list before they can run automatically.

When Bob attempts to run a new command:

- You will get a prompt to approve or reject the command.
- You can choose to add the command pattern to your allowed list.
- Future commands matching that pattern will run automatically.
- This prevents unexpected or malicious commands from executing.

[LLM risk detection](#)

Bob includes AI-powered risk detection that analyzes commands before execution:

- The LLM evaluates each command for potential security risks.
- High-risk commands may require additional confirmation even with auto-approve enabled.
- This provides an extra layer of protection against dangerous operations.
- Risk assessment considers command structure, arguments, and potential system impact.

[AST-based command validation](#)

Bob uses Abstract Syntax Tree (AST) parsing to analyze command structure:

- **Command chaining detection:** Identifies attempts to chain multiple commands using `&&`, `|`, `;`, or `|.`
- **Enhanced security:** Prevents malicious command injection through chaining.
- **Pattern matching:** Validates commands against approved patterns more accurately.
- **Safer auto-approval:** Reduces risk of unintended command execution.

This multi-layered approach ensures that even with auto-approve enabled, dangerous or unexpected commands are caught before execution.

[Question](#)

Bob automatically selects the first suggested answer for a question after a timeout period.

This relates to workflow automation rather than system access.

Todo

Bob can automatically update the todo list without asking for permission.

This allows Bob to track progress and manage task completion as work proceeds. This permission relates to task management rather than system access.

Skills

Bob can automatically activate Skills you have defined without asking for permission.

Skills are reusable instruction sets that teach Bob specialized workflows and tasks. When enabled, Bob will load and follow skill instructions based on your request and the skill's description. The risk level depends on what actions the skill instructions direct Bob to perform, but skills operate within Bob's existing tool constraints.

For more information, see [Skills](#).

Best practices for security

Follow these guidelines to use auto-approving actions safely:

- **Start restrictive:** Begin with minimal permissions and add more only as needed.
- **Use project-specific settings:** Customize auto-approve settings for each project based on risk tolerance.
- **Disable when not needed:** Turn off auto-approve when working with sensitive code or production systems.
- **Review changes regularly:** Periodically check what actions Bob has taken, especially file modifications.
- **Never auto-approve in production:** Restrict auto-approve settings to development environments only.

Why use different modes?

- **Task specialization:** Get precisely the type of assistance you need for your current task
- **Safety controls:** Prevent unintended file modifications when focusing on planning or learning
- **Focused interactions:** Receive responses optimized for your current activity
- **Workflow optimization:** Transition seamlessly between planning, implementing, debugging, and learning

Switching between modes

You have 4 options to switch modes:

- **Drop-down menu:** Click the selector to the left of the chat input
- **Slash command:** Type `/plan`, `/ask`, `/code`, `/advanced`, or `/orchestrator` at the beginning of your message. This will switch to that mode and clear the input field.
- **Toggle command/Keyboard shortcut:** Use the keyboard shortcut below, applicable to your operating system. Each press cycles through the available modes in sequence, wrapping back to the first mode after reaching the end.

Operating System	Shortcut
macOS	⌘ + .

Operating System	Shortcut
Windows	Ctrl + .
Linux	Ctrl + .

- **Accept suggestions:** Click on mode switch suggestions that Bob offers when appropriate

Built-in modes

By default, the following modes are available with Bob:

Code mode

Aspect	Details
Name	 Code
Role definition	You are Bob, a highly skilled software engineer with extensive knowledge in many programming languages, frameworks, design patterns, and best practices.
Short description	Write, modify, and refactor code.
Tool access	read, edit, command
Use case	General purpose coding tasks, optimized for cost efficiency.

[Ask mode](#)

Aspect	Details
Name	? Ask
Role definition	You are Bob, a knowledgeable technical assistant focused on answering questions and providing information about software development, technology, and related topics.
Short description	Get answers and explanations.
Tool access	read, browser, mcp
Use case	Conversational questions and information about your code.

[Plan mode](#)

Aspect	Details
Name	 Plan
Role definition	You are Bob, an experienced technical leader who is inquisitive and an excellent planner. Your goal is to gather information and get context to create a detailed plan for accomplishing the user's task, which the user will review and approve before they switch into another mode to implement the solution.
Short description	Plan and design before implementation.
Tool access	read, edit - markdown only, browser, mcp

Aspect	Details
Use case	High-level planning and technical leadership. Big picture thinking!

[Advanced mode](#)

Aspect	Details
Name	 Advanced
Role definition	You are Bob, a highly skilled software engineer with extensive knowledge in many programming languages, frameworks, design patterns, and best practices.
Short description	Advanced version of Code mode, with more tools.
Tool access	All tool groups: <code>read</code> , <code>edit</code> , <code>command</code> , <code>mcp</code>
Use case	Advanced coding tasks, requiring access to MCP tools. Power users!
Special features	No tool restrictions—full flexibility for all coding tasks

[Orchestrator mode](#)

Aspect	Details
Name	 Orchestrator

Aspect	Details
Role definition	You are IBM Bob, a strategic workflow orchestrator who coordinates complex tasks by delegating them to appropriate specialized modes. You have a comprehensive understanding of each mode's capabilities and limitations, allowing you to effectively break down complex problems into discrete tasks that can be solved by different specialists.
Short description	Coordinate tasks across multiple modes
Tool access	None
Use case	Complex projects requiring coordination across different specialties, multi-domain workflows, and task management
Special features	Automatically switches between modes based on task requirements, manages subtasks, coordinates work across different expertise areas

Comparing the built-in modes

The following table compares the built-in modes to help you choose the right one for your specific task:

Mode	Primary Purpose	When to Use	Key Capabilities	Tool Access
 Code	Writing and modifying code	When implementing features, fixing bugs, or making code improvements	Efficient code generation, refactoring, and debugging	read, edit, command
? Ask	Getting information and explanations	When you need to understand concepts or analyze existing code without making changes	Detailed explanations, code analysis, and technical recommendations	read, browser, mcp

Mode	Primary Purpose	When to Use	Key Capabilities	Tool Access
 Plan	Planning and designing	Before implementation, when you need to think through architecture or create a technical plan	System design, breaking down complex problems, creating specifications	read, edit (markdown only), browser, mcp
 Advanced	Complex development tasks	When you need full access to all tools for sophisticated development workflows	All capabilities of other modes combined with no restrictions	All tool groups
 Orchestrator	Multi-step project coordination	When you have complex projects with multiple moving parts that require different specialties	Task breakdown, mode coordination, workflow management across domains	None

[Mode selection guidelines](#)

- **Start with Plan mode** for new projects or complex features to plan before implementation.
- **Use Code mode** for most day-to-day development tasks.
- **Switch to Ask mode** when you need explanations without modifying files.
- **Use Advanced mode** when you need unrestricted access to all tools for complex workflows.
- **Use Orchestrator mode** for complex, multi-step projects that require coordination across different specialties and domains.

Customizing modes

Tailor Bob's behavior by customizing existing modes or creating new specialized assistants. Define tool access, file permissions, and behavior instructions to enforce team standards or create purpose-specific assistants.

For setup instructions, see [Custom modes](#).

Types of mentions

With context mentions, you can reference various elements of your project:

Mention type	Format	Description	Example usage
File	@/path/to/file.ts	Includes file contents in request context	Explain the function in @/src/utils.ts
Folder	@/path/to/folder	Includes contents of all files directly in the folder (non-recursive)	Analyze the code in @/src/components
Problems	@problems	Includes IBM Bob Problems panel diagnostics	@problems Fix all errors in my code
Terminal	@terminal	Includes recent terminal command and output	Fix the errors shown in @terminal
Git commit	@a1b2c3d	References specific commit by hash	What changed in commit @a1b2c3d?
Git changes	@git-changes	Shows uncommitted changes	Suggest a message for @git-changes
URL	@https://example.com	Imports website content	Summarize @https://docusaurus.io/

File mentions

File mentions are the most commonly used type, letting you include the contents of specific files in your conversation with Bob.

Capability	Details
Format	@/path/to/file.ts (always start with / from workspace root)
Provides	Complete file contents with line numbers
Works with	Text files, PDFs, and DOCX files (with text extraction)
Works in	Initial requests, feedback responses, and follow-up messages
Limitations	Very large files may be truncated; binary files not supported

Tip:

For large files, you can mention specific sections by using the file's path followed by line numbers, like @/src/app.js:10-20 to include only lines 10-20.

[Folder mentions](#)

With folder mentions, you can reference multiple files at once, providing broader context from a directory.

Capability	Details
Format	@/path/to/folder (no trailing slash)

Capability	Details
Provides	Complete contents of all files within the directory
Includes	Contents of non-binary text files directly within the folder (not recursive)
Best for	Providing context from multiple files in a directory
Tip	Be mindful of context window limits when mentioning large directories

Note:

Folder mentions are non-recursive, meaning they only include files directly in the specified folder, not in subfolders.

[Problems mention](#)

The problems mention imports diagnostics from IBM Bob's Problems panel, helping you resolve multiple issues at once.

Capability	Details
Format	@problems
Provides	All errors and warnings from IBM Bob's problems panel
Includes	File paths, line numbers, and diagnostic messages
Groups	Problems organized by file for better clarity
Best for	Fixing errors without manual copying

Example usage: @problems Can you help me fix all these issues?

Terminal mention

Terminal mentions capture recent command output, helping you debug build errors or analyze command results.

Capability	Details
Format	@terminal
Captures	Last command and its complete output
Preserves	Terminal state (does not clear the terminal)
Limitation	Limited to visible terminal buffer content
Best for	Debugging build errors or analyzing command output

Tip

When troubleshooting a build failure, try `What is wrong with @terminal?` to get immediate insights into the error without manually copying the output.

[Git mentions](#)

Git mentions help you work with version control by referencing commits and changes.

Type	Format	Provides	Limitations
Commit	@a1b2c3d	Commit message, author, date, and complete diff	Only works in Git repositories
Working changes	@git-changes	git status output and diff of uncommitted changes	Only works in Git repositories

Example usage: What's the purpose of @a1b2c3d? **or** Suggest a commit message for @git-changes

[URL mentions](#)

URL mentions let you reference external web content in your conversations with Bob.

Capability	Details
Format	@https://example.com
Processing	Uses headless browser to fetch content
Cleaning	Removes scripts, styles, and navigation elements
Output	Converts content to Markdown for readability

Capability	Details
Limitation	Complex pages may not convert perfectly

Tip:

URL mentions are useful for referencing documentation. Try `Can you explain @https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise` to get help understanding a concept with the official docs as context.

Use context mentions

Type `@` in the chat input to trigger the suggestions dropdown.

Continue typing to filter suggestions or use arrow keys to navigate.

Select with Enter key or mouse click.

Combine multiple mentions in a request: "Fix @problems in @/src/component.ts"

Tip:

Highlight text and use the keyboard shortcut `⌘ + L` to add it to the chat.

The dropdown automatically suggests:

- Recently opened files
- Visible folders
- Recent git commits
- Special keywords (`problems`, `terminal`, `git-changes`)
- All currently open files (regardless of ignore settings or directory filters)

The dropdown automatically filters out common directories like `node_modules`, `.git`, `dist`, and `out` to reduce noise, even though you can include their content if manually typed.

Tip:

You can combine multiple mentions in a single request. Try `Compare @/src/v1/api.js with @/src/v2/api.js` and explain the differences.

Important behaviors

Ignore file interactions

Context mentions have specific behaviors when interacting with ignored files:

Behavior	Description
.bobignore bypass	File and folder <code>@mentions</code> bypass <code>.bobignore</code> checks when fetching content for context. Content from ignored files will be included if directly mentioned.
.gitignore bypass	Similarly, file and folder <code>@mentions</code> do not respect <code>.gitignore</code> rules when fetching content.
Git command respect	Git-related mentions (<code>@git-changes</code> , <code>@commit-hash</code>) do respect <code>.gitignore</code> since they rely on Git commands.

Context mentions bypass ignore rules, so you can reference specific files that would normally be ignored by Bob or Git. This capability is particularly useful when you need to discuss generated files or other typically ignored content.

Putting it all together

Context mentions enhance your interactions with Bob by providing precise references to your project's elements. Instead of copying and pasting code or describing file locations, you can directly reference what you are discussing.

Effective combinations of different mention types:

- Fix `@problems` in `@/src/components` and explain what was wrong
- Review `@git-changes` and suggest improvements
- Compare `@/v1/api.js` with `@https://api-docs.example.com` and check for inconsistencies

By using context mentions effectively, you can communicate more precisely with Bob and receive more targeted assistance.

Why use Bob tips?

- **Real-time code analysis:** Automatically detects quality issues as you write code, without manual scanning.
- **Catch quality issues early:** Identify problematic functions during development, before code review.
- **Reduce technical debt:** Address complexity and maintainability issues as they emerge.
- **Improve code readability:** Get specific suggestions for simplifying complex logic.
- **Enhance testability:** Reduce deeply nested logic and excessive branching that makes testing difficult.

How it works

Bob tips performs continuous static analysis on all open files in your editor, calculating code quality metrics at the function level:

Metrics analyzed:

- **Cyclomatic complexity:** Measures the number of independent paths through your code. A value of 10 or higher is considered high complexity.

- **Maintainability index:** Evaluates how easy your code is to read, test, and extend. A value of 70 or lower is considered low maintainability.

Automatic detection:

When a function exceeds quality thresholds, Bob automatically:

1. Marks the function with a purple underline in your editor.
2. Adds the finding to the Bob Findings panel.
3. Generates AI-powered refactoring suggestions.

Bob tips runs continuously in the background without requiring manual scans or configuration. Analysis happens automatically as you write and modify code.

What Bob tips detects

Bob tips specifically targets two categories of code quality issues:

High cyclomatic complexity

Functions with deeply nested logic or excessive branching:

- Multiple nested if/else statements.
- Complex switch statements with many cases.
- Deeply nested loops.
- Excessive conditional logic.

These patterns make code difficult to understand, test, and maintain.

Low maintainability

Functions that are difficult to read, test, or extend:

- Long functions with too many responsibilities.
- Poor separation of concerns.
- Complex logic that is hard to follow.
- Code that would benefit from decomposition.

View and fix findings

In the editor

When Bob identifies a quality issue:

1. The problematic function appears with a purple underline in your editor.
2. Hover over the underline to see a tooltip with:
 - The specific quality issue detected.
 - AI-powered refactoring suggestions.
 - Options to discuss the issue with Bob.

Using the Bob Findings panel

All findings across your open files are aggregated in the Bob Findings panel:

- View all quality issues in one centralized location.
- Navigate between findings across multiple files.
- Track which issues you have addressed.

- Reference findings in chat using '@problems'. For more information, see [context mentions](#).

Fix with Bob

Collaborate with Bob to refactor your code:

Hover over the purple underline in your editor.

Click **Fix with Bob** in the tooltip.

Bob opens a chat conversation with context about the issue.

Discuss the refactoring approach and implement the solution together.

Use this option when you want to:

- Understand the issue in more detail.
- Explore different refactoring approaches.
- Plan a more complex refactoring strategy.
- Learn best practices for similar situations.

Configure Bob tips

Enable or disable the feature

Click the **Bob - Settings** button in the bottom right corner of IBM Bob.

Click the **Bob findings** tab.

Toggle **Bob tips** on or off.

When disabled, Bob stops analyzing files and removes existing findings from the editor and Bob Findings panel.

Overview

To get started, type `/` in the chat to see all available commands, or create your own by adding a markdown file to `.bob/commands/` or `~/ .bob/commands/`.

Key benefits:

- Workflow automation: Turn complex multi-step processes into single commands
- Team standardization: Share commands across your team for consistent practices
- Context preservation: Include project-specific context in every command
- Quick access: Fuzzy search and autocomplete for instant command discovery

Built-in commands

Bob includes several built-in commands that provide core functionality:

[/init](#)

Initialize a new project or workspace with Bob. This command helps set up the necessary configuration and structure for working with Bob in your project.

[/review](#)

Review code changes with comprehensive analysis. This command can be used in multiple ways:

- `/review` - Review local uncommitted changes in your working directory
- `/review <branch>` - Compare a branch against your current branch (HEAD)
- `/review #<issue-number> --issue-coverage` - Validate local changes against a GitHub issue
- `/review <issue-url> --issue-coverage` - Validate local changes against a GitHub issue URL

The review command performs thorough code analysis including bug detection, security checks, performance issues, and style consistency.

[/create-pr](#)

Create a pull request with AI-generated description. This command analyzes your changes and generates a comprehensive PR description based on the diff between branches.

Creating custom commands

Custom commands extend Bob's functionality by adding markdown files to specific directories:

- Project-specific: `.bob/commands/` in your workspace root
- Global: `~/ .bob/commands/` in your home directory

The filename becomes the command name. For example:

- `review.md` → `/review`
- `test-api.md` → `/test-api`
- `deploy-check.md` → `/deploy-check`

Command name processing

When creating commands through the UI, command names are automatically processed:

- Converted to lowercase
- Spaces replaced with dashes
- Special characters removed
- Leading/trailing dashes removed

Example: "My Cool Command!" becomes `my-cool-command`

Basic command format

Create a simple command by adding a markdown file:

```
Help me review this code for security issues and suggest improvements.
```

[Advanced command with frontmatter](#)

Add metadata using frontmatter for enhanced functionality:

```
---
description: Create a new API endpoint
argument-hint: <endpoint-name> <http-method>
---
Create a new API endpoint called $1 that handles $2 requests.
Include proper error handling and documentation.
```

[Frontmatter fields](#)

- `description`: Appears in the command menu to help users understand the command's purpose
- `argument-hint`: Provides a hint about expected arguments when using the command

[Command management UI](#)

Bob provides a dedicated UI for managing custom commands.

Click the commands icon in the Bob panel to open the command manager.

Creating a new command

1. Type your command name in the input field (e.g., "Sample command name")
2. Click the + button to create the command
3. A new file will be created and opened automatically (e.g., `sample-command-name.md`)

Using slash commands

Type `/` in the chat to see a unified menu containing both types of commands. The menu shows both custom workflow commands and mode-switching commands in the same interface.

1. Unified menu: Both custom commands and mode-switching commands appear together
2. Autocomplete: Start typing to filter commands (e.g., `/sam` shows `sample-command-name`)
3. Fuzzy search: Find commands even with partial matches
4. Description preview: See command descriptions in the menu
5. Visual indicators: Mode commands are distinguished from custom commands with special icons

[Argument hints](#)

Argument hints provide instant help for slash commands, showing you what kind of information to provide when a command expects additional input.

When you type `/` to bring up the command menu, commands that expect arguments will display a light gray hint next to them. This hint tells you what kind of argument the command is expecting.

For example:

- `/mode <mode_slug>` - The hint `<mode_slug>` indicates you should provide a mode name like `code` or `debug`
- `/api-endpoint <endpoint-name> <http-method>` - Shows you need both an endpoint name and HTTP method

After selecting the command, it will be inserted into the chat input followed by a space. The hint is not inserted; it is only a visual guide to help you know what to type next. You must then manually type the argument after the command.

[Adding argument hints to custom commands](#)

You can add argument hints to your custom commands using the `argument-hint` field in the frontmatter:

```
---  
description: Create a new API endpoint  
argument-hint: <endpoint-name> <http-method>  
---  
Create a new API endpoint called $1 that handles $2 requests.
```

This will display as `/api-endpoint <endpoint-name> <http-method>` in the command menu.

Best practices for argument hints

- Be specific: Use descriptive placeholders like `<file-path>` instead of generic ones like `<arg>`
- Show multiple arguments: If your command needs multiple inputs, show them all: `<source>`
`<destination>`
- Use consistent format: Always wrap placeholders in angle brackets: `<placeholder>`
- Keep it concise: Hints should be brief and clear

Common questions

- "What if I do not provide the argument?" The command might not work as expected, or it might prompt you for more information. The hint is there to help you get it right the first time.
- "Do all commands have hints?" No, only commands that are designed to take arguments will have hints. Commands that work without additional input will not show hints.

- "Can I use a command without replacing the hint?" The hint text (like `<mode_slug>`) needs to be replaced with actual values. Leaving the hint text will likely cause the command to fail or behave unexpectedly.

Best practices

Command naming

- Use descriptive, action-oriented names
- Keep names concise but clear
- Use hyphens for multi-word commands
- Avoid generic names like `help` or `test`
- Note: Names are automatically slugified (lowercase, special characters removed)
- The `.md` extension is automatically added/removed as needed

Command content

- Start with a clear directive

- Use structured formats (lists, sections)
- Include specific requirements
- Reference project conventions
- Keep commands focused on a single task

Organization

- Group related commands in subdirectories
- Use consistent naming patterns
- Document complex commands
- Version control your commands
- Share team commands in the project repository

Troubleshooting

Commands not appearing

- Check file location: Ensure custom command files are in `.bob/commands/` or `~/ .bob/commands/`
- Verify file extension: Custom commands must be `.md` files

Command not found

When a slash command is not found, the LLM will see:

```
The slash command '/unknown-command' was not found. Please check the command name and try again.
```

Command conflicts

- Custom project commands override global custom commands with the same name
- Use unique names to avoid conflicts
- When creating duplicate names through the UI, numbers are appended (e.g., `new-command-1`, `new-command-2`)

About mode commands

The slash menu includes mode-switching commands (like `/code`, `/ask`) that fundamentally change the AI's operational mode - they do not just inject text but switch the entire AI context. Custom modes you create also appear as slash commands (e.g., a mode with slug `reviewer` becomes `/reviewer`). These mode commands cannot be overridden by custom workflow commands.

Learn more in [Modes](#) and [Custom modes](#).

What are checkpoints

Checkpoints let you:

- Safely experiment with AI-suggested changes
- Easily recover from undesired modifications
- Compare different implementation approaches
- Revert to previous project states without losing work

How checkpoints work

Bob captures snapshots of your project's state using a shadow Git repository, separate from your main version control system. These snapshots, called checkpoints, are automatically created before file modifications occur, ensuring you can always undo unwanted changes.

Checkpoints are recorded when tasks begin and before file modifications. They are not automatically created before running commands.

Working with checkpoints

Checkpoints are integrated directly into your workflow through the [chat interface](#) and appear directly in your chat history:

- Task checkpoint marks your starting project state.
- Regular checkpoints are created before file modifications, allowing easy undo of any changes.

Each checkpoint provides two primary functions that are always available in both the chat history and checkpoint menu:

Viewing differences

To compare your current workspace with a previous checkpoint:

1. Locate the checkpoint in your chat history or open the checkpoint menu.
2. Click the checkpoint's View Differences button.
3. Review the differences in the comparison view:
 - Added lines are highlighted in green.
 - Removed lines are highlighted in red.
 - Modified files are listed with detailed changes.
 - Renames may not always be detected; diffs reflect per-file changes between commits.
 - New or deleted files are clearly marked.

[Restoring checkpoints](#)

Restore options are always visible in checkpoint buttons and dialogs, regardless of whether changes are detected. This ensures you can always revert to any checkpoint state.

To restore a project to a previous checkpoint state:

1. Locate the checkpoint in your chat history or checkpoint menu.
2. Click the checkpoint's Restore Checkpoint button.
3. Choose one of these restoration options:

Restore files: Reverts only workspace files to checkpoint state without modifying conversation history. Ideal for comparing alternative implementations while maintaining chat context, allowing you to seamlessly switch between different project states. This option does not require confirmation and lets you quickly switch between different implementations.

Restore files & task: Reverts both workspace files AND removes all subsequent conversation messages. Use when you want to completely reset both your code and conversation back to the checkpoint's point in time. This option requires confirmation in a dialog as it cannot be undone.

[Limitations and considerations](#)

- **Scope:** Checkpoints only capture changes made during active Bob tasks.
- **External changes:** Modifications made outside of tasks (manual edits, other tools) are not included.
- **Large files:** Very large binary files may impact performance.
- **Unsaved work:** Restoration will overwrite any unsaved changes in your workspace.
- **Timing:** Checkpoints are created before changes are applied, providing a safety net for all modifications.
- **Memory usage:** Checkpoints accumulate over time and can consume significant memory, especially for large projects or long-running tasks. If you have many old tasks with checkpoints, consider clearing them periodically to free up memory. You can do this by deleting old task conversations, which will remove their associated checkpoints.

Technical implementation

Checkpoint architecture

The checkpoint system consists of:

1. **Shadow Git repository:** A separate Git repository created specifically for checkpoint tracking that functions as the persistent storage mechanism for checkpoint state.
2. **Checkpoint service:** Handles Git operations and state management through:
 - Repository initialization
 - Checkpoint creation and storage

- Diff computation
- State restoration

3. **UI components:** Interface elements displayed in the chat that enable interaction with checkpoints.

Restoration process

When restoration runs, Bob:

- Completes a reset to the specified checkpoint commit.
- Copies all files from the shadow repository to your workspace.
- Updates internal checkpoint tracking state.

Storage type

Checkpoints are task-scoped, meaning they are specific to a single task.

Diff computation

Checkpoint comparison uses Git's underlying diff capabilities to produce structured file differences:

- Modified files show line-by-line changes.
- Binary files are properly detected and handled.
- Rename detection may be limited; diffs focus on file content changes between checkpoints.
- File creation and deletion are clearly identified.

File exclusion and ignore patterns

The checkpoint system uses intelligent file exclusion to track only relevant files:

Built-in exclusions

The system has comprehensive built-in exclusion patterns that automatically ignore:

- Build artifacts and dependency directories (`node_modules/`, `dist/`, `build/`).
 - Media files and binary assets (images, videos, audio).
 - Cache and temporary files (`.cache/`, `.tmp/`, `.bak`).
 - Configuration files with sensitive information (`.env`).
 - Large data files (archives, executables, binaries).
-

- Database files and logs.

These patterns are written to the shadow repository's `.git/info/exclude` file during initialization.

[.gitignore support](#)

The checkpoint system respects `.gitignore` patterns in your workspace:

- Files excluded by `.gitignore` will not trigger checkpoint creation.
- Excluded files will not appear in checkpoint diffs.
- Standard Git ignore rules apply when staging file changes.

[Git LFS patterns](#)

Patterns defined in your workspace's `.gitattributes` for Git LFS are read and added to checkpoint exclusions. This helps avoid tracking large LFS-managed assets in the shadow repository.

[.bobignore behavior](#)

The `.bobignore` file (which controls AI access to files) is separate from checkpoint tracking:

- Files excluded by `.bobignore` but not by `.gitignore` will still be checkpointed.
- Changes to AI-inaccessible files can still be restored through checkpoints.

This separation is intentional, as `.bobignore` limits which files the AI can access, not which files should be tracked for version history.

[Disabling checkpoints](#)

You can enable or disable checkpoints in the Bob settings panel under the Checkpoints section:

1. Open Settings by clicking the gear icon.
2. Click the `Chat` tab and find Checkpoints near the bottom of the panel.

[Nested Git repositories](#)

If nested Git repositories are detected in your workspace, checkpoints are disabled. You'll receive a clear notification when checkpoint initialization fails due to nested repositories. Remove or relocate nested repositories to enable checkpoints.

[Concurrency control](#)

The extension prevents duplicate checkpointing within a single streaming operation. There is no dedicated Git operation queue.

[Git installation](#)

Checkpoints require Git to be installed on your system. The implementation uses the simple-git library, which relies on Git command-line tools to create and manage shadow repositories.

Install Git with Homebrew (recommended)

```
brew install git
```


Alternative: Install with Xcode Command Line Tools:

```
xcode-select --install
```

Verify installation

- Open Terminal
- Type `git --version`
- You should see a version number like `git version 2.40.0`

Download Git for Windows

- Visit <https://git-scm.com/download/win> and the download should start automatically

Run the installer

- Accept the license agreement
- Choose installation location (default is recommended)
- Select components (default options are typically sufficient)
- Choose the default editor
- Choose how to use Git from the command line (recommended: Git from the command line and also from 3rd-party software)
- Configure line ending conversions (recommended: Checkout Windows-style, commit Unix-style)
- Complete the installation

Verify installation

- Open Command Prompt or PowerShell
- Type `git --version`
- You should see a version number like `git version 2.40.0.windows.1`

Install Git using your distribution's package manager:

• Debian/Ubuntu:

```
sudo apt-get update  
sudo apt-get install git
```

• Fedora:

```
sudo dnf install git
```

• Arch Linux:

```
sudo pacman -S git
```

Verify installation

- Open Terminal
- Type `git --version`
- You should see a version number

Why use code actions?

- **Quick access to Bob:** Add code to your conversation without copying and pasting
- **Contextual assistance:** Get help with the exact code you are working on
- **Streamlined workflow:** Access Bob's capabilities without leaving your editor
- **Automatic context:** Bob receives file paths and line numbers automatically

How it works

Code actions appear when you:

- Select a range of code
- Place your cursor on a line with a diagnostic (error, warning, or hint)
- Invoke them through a keyboard shortcut or command

A lightbulb icon () appears in the editor gutter (the area to the left of line numbers) when code actions are available. You can also access them through the right-click context menu or keyboard shortcut (`Ctrl + .` or `Cmd + .` on macOS).

Available code actions

Bob provides these code actions:

Action	Description	Position in menu
Add to Context	Adds selected code to your Bob conversation with file path and line numbers	First (for quick access)
Explain Code	Asks Bob to explain the selected code	Second
Improve Code	Asks Bob to suggest improvements to the selected code	Third
Inline Chat	Opens an inline chat interface directly in the editor at the cursor position for quick interactions	Fourth
Move to Chat	Sends the selected code to the Bob chat panel with context	Fifth

[Add to Context](#)

The Add to Context action appears first in the menu for quick access. When you use it, Bob includes the filename and line numbers with your code, helping Bob understand the exact location within your project.

Example of what Bob receives:

```
Can you explain this function?  
@myFile.js:15:25
```

The `@myFile.js:15:25` notation represents the code you added, including its location.

[Inline Chat](#)

The Inline Chat action opens a chat interface directly in your editor at the cursor position, allowing you to interact with Bob without switching to the chat panel. This is useful for quick questions or edits while staying focused on your code.

How to use:

1. Select the code you want to discuss or place your cursor where you want to chat.
2. Trigger the Inline Chat action through the lightbulb menu, context menu, or keyboard shortcut (default: `Cmd + K` on macOS, `Ctrl + K` on Windows/Linux).

3. Type your question or request in the inline chat interface.
4. Bob responds directly in the editor, and you can accept, reject, or modify the suggestions.

[Move to Chat](#)

The Move to Chat action sends your selected code to the Bob chat panel with full context, including file path and line numbers. This is ideal when you want to have a more detailed conversation about the code in the main chat interface.

How to use:

1. Select the code you want to discuss.
2. Trigger the Move to Chat action through the lightbulb menu, context menu, or keyboard shortcut (default: `Cmd + L` on macOS, `Ctrl + L` on Windows/Linux).
3. The selected code is automatically added to the chat panel with context.
4. Continue the conversation in the chat panel with Bob.

[Using code actions](#)

You can access Bob's code actions in three ways:

From the lightbulb icon

1. Select the code you want to work with.
- 2.

Click the lightbulb icon () that appears in the gutter.

3. Select the Bob action you want from the menu.
4. Review Bob's response in the chat panel.

From the context menu

1. Select the code you want to work with.
2. Right-click the selected code.
3. Select **Bob** from the context menu.
4. Select the action you want from the submenu.
5. Review Bob's response in the chat panel.

From the Command Palette

1. Select the code you want to work with.
2. Open the Command Palette with `Ctrl+Shift+P` (Windows/Linux) or `Cmd+Shift+P` (macOS).
3. Type "Bob" to filter commands.
4. Select the code action you want (for example, "Bob: Explain Code").
5. Review Bob's response in the chat panel.

Bob's tip: The Command Palette applies actions to your current active chat task if one exists.

What is Next Edit?

Get code completions and automatic error fixes as you work in your editor. When enabled, Bob analyzes your code context and suggests edits through an inline diff interface.

Key capabilities

- **Code completions:** Suggests the next change to your codebase based on your code context, intent, and logic.
- **Automatic error fixes:** Detects diagnostic errors near your cursor and suggests fixes.
- **Inline diff interface:** Displays suggestions with jump decorations and gutter indicators.

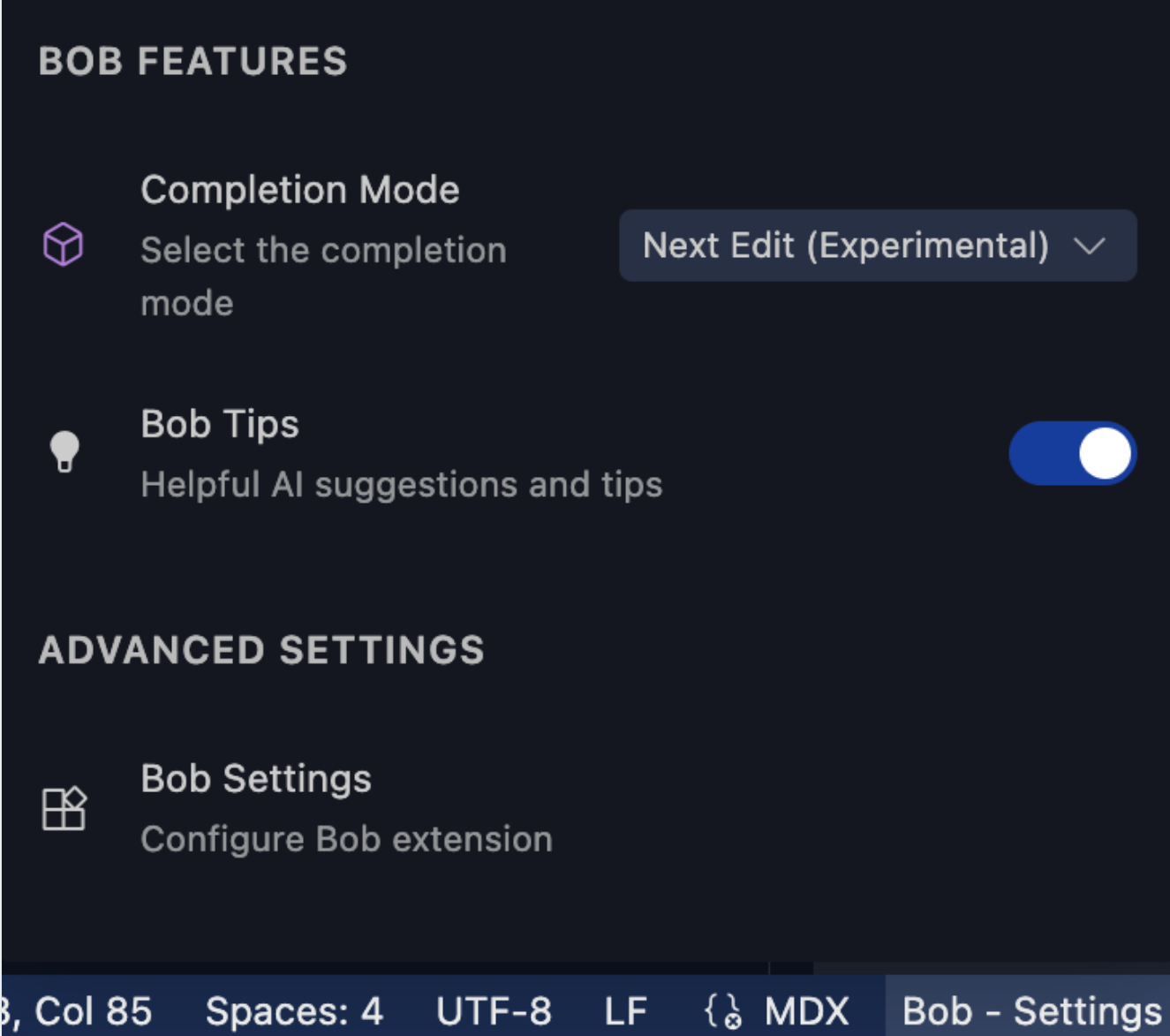
Enabling Next Edit

You can enable Next Edit through Quick Settings or the Settings menu.

[From Quick Settings](#)

Click **Bob - Settings** in the status bar at the bottom of IBM Bob.

Select **Next Edit (Experimental)** from the Completion Mode dropdown.



The screenshot displays the IBM Bob settings interface. It is divided into two main sections: 'BOB FEATURES' and 'ADVANCED SETTINGS'. In the 'BOB FEATURES' section, there are two items: 'Completion Mode' with a description 'Select the completion mode' and a dropdown menu currently showing 'Next Edit (Experimental)'; and 'Bob Tips' with a description 'Helpful AI suggestions and tips' and a toggle switch that is turned on. The 'ADVANCED SETTINGS' section contains one item: 'Bob Settings' with a description 'Configure Bob extension'. At the bottom of the panel is a status bar showing various editor settings: '8, Col 85', 'Spaces: 4', 'UTF-8', 'LF', a file icon, 'MDX', and 'Bob - Settings'.

BOB FEATURES

- Completion Mode**
Select the completion mode
Next Edit (Experimental) ▾
- Bob Tips**
Helpful AI suggestions and tips
☒

ADVANCED SETTINGS

- Bob Settings**
Configure Bob extension

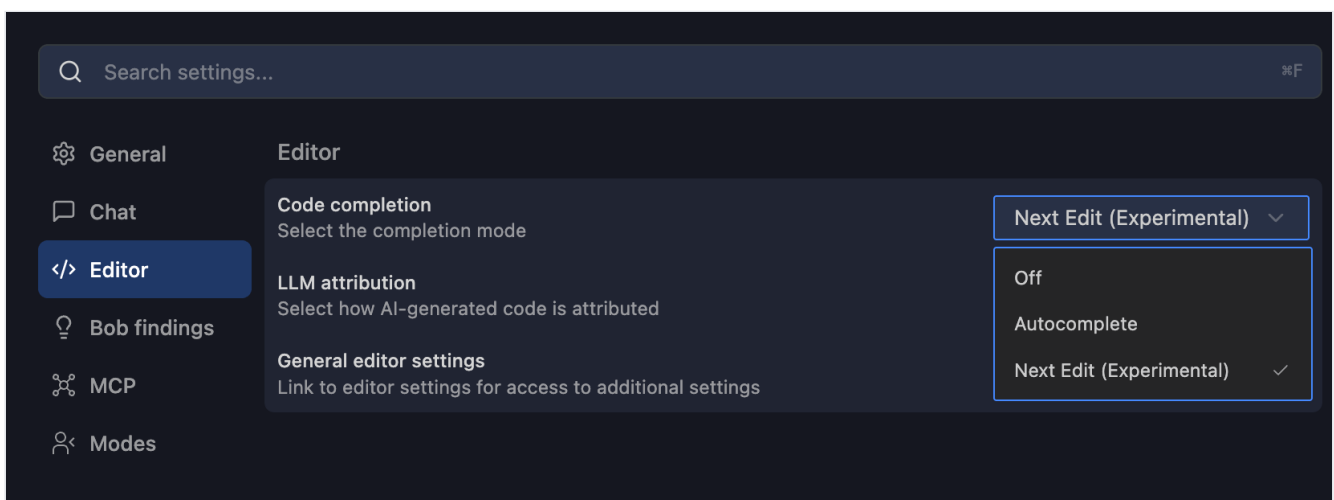
8, Col 85 Spaces: 4 UTF-8 LF { } MDX Bob - Settings

[From Settings menu](#)

Click the settings gear icon in the Bob panel.

Select the **Editor** tab.

Select **Next Edit (Experimental)** from the Completion Mode dropdown.



[Using Next Edit](#)

Position your cursor in your code where you want to make changes.

Wait for Bob to analyze the context and display a suggestion as an inline diff.

Review the suggested changes shown with visual indicators in the editor gutter.

Accept the suggestion using the keyboard shortcut or by clicking the accept indicator, or reject it to continue coding.

Learn more about the Next Edit feature in the [Get code predictions](#) tutorial.

Why use code reviews?

- **Catch errors early:** Identify issues before they make it into your committed work.
- **Save time:** Address potential reviewer comments proactively, reducing PR approval time.
- **Improve code quality:** Get suggestions for better coding practices and maintainability.
- **Validate against issues:** Ensure your changes align with issue requirements (GitHub required).

How it works

Bob provides a dedicated Review Panel in your IDE's sidebar where you can configure and initiate code reviews. The panel offers two review modes:

1. **Branch Comparison:** Compare your changes against any branch.
2. **Issue Coverage:** Validate your local changes against a specific GitHub issue.

Bob analyzes your changes and flags potential issues in the Bob Findings panel, which provides:

- **Hierarchical file lists:** Navigate large changesets more easily with organized file structures.
- **Remote branch support:** Review work from any branch, not just your current one.

- **Issue alignment validation:** Verify that your changes match the intent and requirements of an issue (GitHub required).
- **Configurable exclusions:** Exclude specific files or patterns from reviews.

[Review your code](#)

[Prerequisites](#)

- You have uncommitted changes in your workspace.
- Access to remote or local branches you want to review.
- For issue coverage validation: A GitHub account and issue URL (GitHub is required for issue-related features).

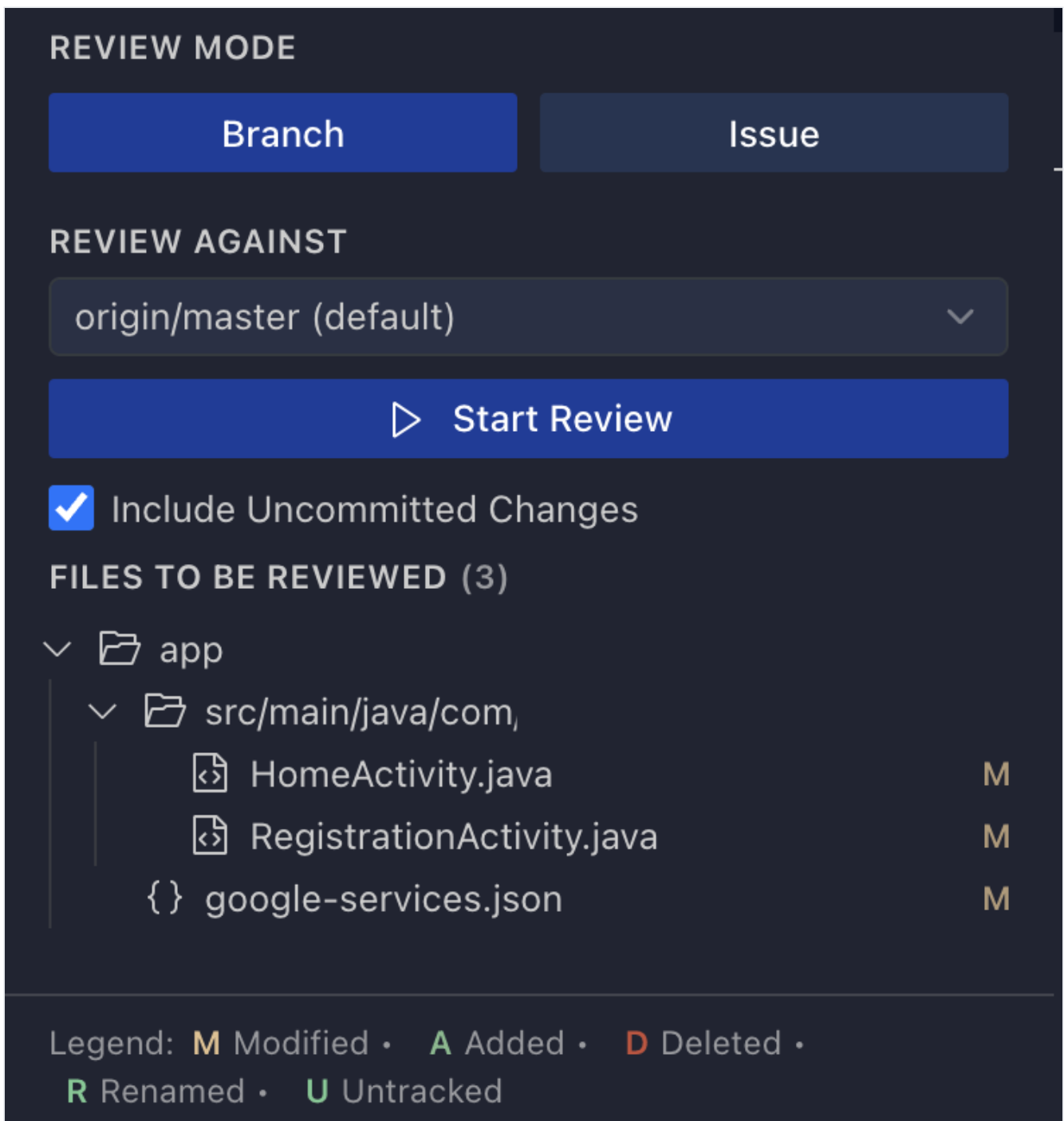
Note:

Bob can work with GitHub and GitLab for pull request workflows and branch comparisons. However, issue coverage validation requires GitHub.

Initiate a review

You can initiate a code review using the Review Panel or chat commands.

Using the Review Panel



Open the Bob Review panel from your sidebar. The panel provides a visual interface with two review modes:

Branch:

Select Branch at the top of the panel.

Choose a branch to compare against from the dropdown.

Toggle Include Uncommitted Changes to include local modifications for non-current branches.

Click Start Review to begin the review.

Bob reviews your changes and displays findings in the Bob Findings panel.

Issue:

Select Issue mode at the top of the panel.

Choose a GitHub issue from the dropdown, or select Other to enter a custom issue URL.

Click Start Review to review your changes against the issue.

Bob analyzes whether your local changes align with the issue's requirements.

Using chat commands

You can initiate reviews directly through the chat interface using slash commands. This is a faster, keyboard-driven alternative to the Review Panel:

Branch comparison:

- `/review` - Review uncommitted changes in your working directory.
- `/review <branch>` - Compare your changes against a specific branch.

Issue coverage:

- `/review <issue-url> --issue-coverage` - Validate changes against a GitHub issue URL.
 - **Example:** `/review https://github.com/owner/repo/issues/123 --issue-coverage`

For more information, see [slash commands](#).

[Review options](#)

[Select a comparison branch](#)

The branch dropdown selector shows:

- **Current branch:** Your active branch with uncommitted changes.
- **Default branch:** Your repository's default branch (marked with a badge).
- **Remote branches:** Branches from your remote repository (marked with a badge).
- **Local branches:** Other local branches (marked with a badge).

[Show uncommitted changes](#)

When reviewing against a branch other than your current branch, enable the Include Uncommitted Changes checkbox to include uncommitted changes in your review. This option is automatically enabled when reviewing your current branch.

[Configure review exclusions](#)

You can exclude specific files or patterns from code reviews:

Open Bob Settings.

Navigate to the Bob Findings tab.

Under Review Exclusions, add glob patterns for files to exclude (e.g., `.vscode/**`, `*.test.ts`).

Excluded files will not appear in the review panel's file list and will not be analyzed during reviews.

[Auto-approval during reviews](#)

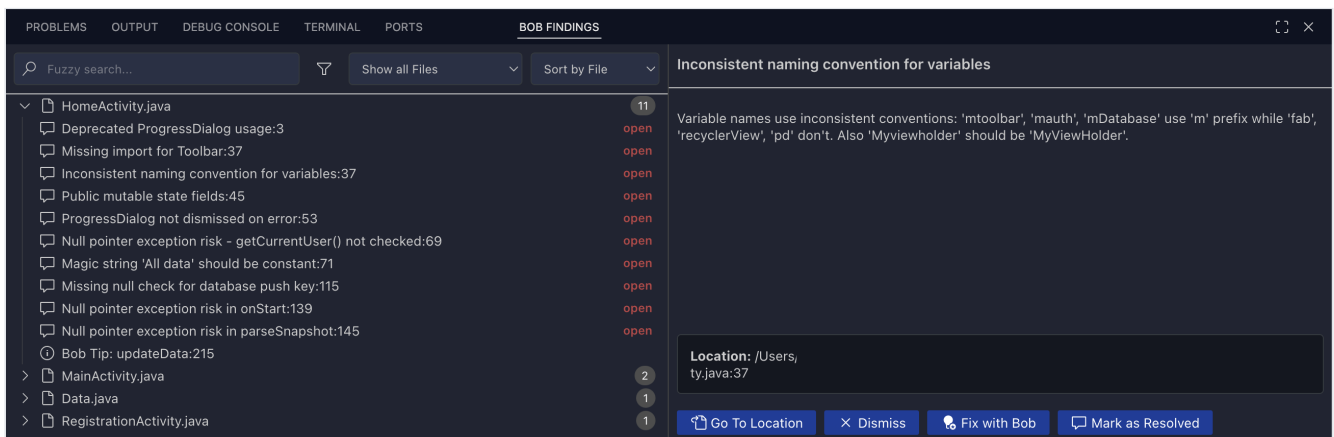
You can enable auto-approval for review actions to keep your workflow moving. This allows Bob to proceed with review-related actions without requiring manual approval for each step.

The review process uses both read tools for analyzing code and write tools for submitting findings. To enable full auto-approval for reviews:

- Enable Always allow read tools for code analysis actions.
- Enable Always allow write tools for submitting review findings to the Bob Findings panel.

For more information about configuring auto-approval, see [auto-approving actions](#).

Using Bob Findings



Once Bob completes a review, findings appear in the Bob Findings panel. You can interact with these findings in several ways:

[Viewing findings](#)

- Click on any finding in the Bob Findings panel to see full details.
- Use `@issues` in the chat interface context dropdown menu to reference findings in conversation.
- Navigate to the file by clicking on the finding to see the code in context with inline annotations.

For more information about context mentions, see [context mentions](#).

[Taking action on findings](#)

Each finding provides several action buttons:

- **Fix with Bob:** Ask Bob to automatically fix the issue. Bob will create a task to address the finding.
- **Mark as Resolved:** Mark the finding as resolved if you've addressed it manually.
- **Mark as Open:** Reopen a previously resolved finding.
- **Dismiss:** Remove the finding from the panel if it's not relevant.

[Finding status tracking](#)

Findings can have three statuses:

- **Open:** Issue identified and needs attention.
- **In Progress:** Bob is actively working on fixing the issue.
- **Resolved:** Issue has been addressed.

[Tips and best practices](#)

- **Combine with manual reviews:** Use Bob's automated reviews as a first pass before human code reviews.
- **Review before committing:** Run reviews on uncommitted changes to catch issues early.
- **Validate issue coverage:** Use Issue mode to ensure your changes fully address the requirements before creating a pull request.
- **Configure exclusions:** Exclude test files, generated code, or configuration files that don't need review.
- **Compare against remote branches:** Use the branch selector to review your work against the latest remote changes.

Why use this feature?

- **Save time:** Generate well-formatted commit messages with a single click
- **Maintain consistency:** Follow conventional commit standards automatically
- **Match project style:** Bob analyzes your branch name and commit history to match your project's conventions
- **Iterate easily:** Generate alternative suggestions if you do not like the first one

How it works

Bob examines your staged changes to understand what modifications you have made to your codebase. It then analyzes your branch name and recent commit history to determine the appropriate commit message format that matches your project's style.

Getting started

Prerequisites

Your changes must be staged in the Source Control panel.

Basic usage

To automatically generate a commit message:

- 1.

Click the sparkle () icon next to the commit message box.

2. Review your commit message made by Bob.

◦

Click the sparkle () icon again for alternative suggestions. Apply edits to your message, if needed.

3. Commit your changes.

Tips and best practices

- For the best results, make atomic commits (changes that address a single concern).
- If you are working on a specific issue, include the issue number in your branch name.
- Edit the generated message if needed to add more specific details.
- Use the regenerate option if the first suggestion does not capture your changes accurately.

Why use this feature?

- **Save time:** Generate detailed PR descriptions automatically based on your changes.
- **Maintain consistency:** Ensure all PRs follow a consistent format.
- **Improve collaboration:** Provide clear context for reviewers with well-structured descriptions.
- **Streamline workflow:** Create PRs without leaving your IDE.

How it works

Bob analyzes your branch changes, commit history, and branch name to understand the purpose and scope of your work. It then generates a meaningful PR title and detailed description that summarizes your changes, making it easier for reviewers to understand the context and purpose of your PR. After generation, you can review and edit the PR description before submission.

The PR generation process is interactive, allowing you to select the target branch and remote repository before creating the PR.

Getting started

Prerequisites

- Your changes must be committed to a branch.
- You must have push access to the remote repository.

Basic usage

You can choose from 3 options to initiate a PR flow:

- Click the PR icon in the Source Control panel.
- Use the command palette (`Cmd + Shift + P` on Mac, `Ctrl + Shift + P` on Windows/Linux) and search for "Bob: Generate PR".
- Type `/create-pr` in the Bob chat interface.

Bob will then guide you through the following steps:

1. Select the base branch you want to merge into. Bob will suggest the three most likely branches.
2. Select the remote repository, if you have multiple remotes.
3. Review the generated PR description and title.
4. Create the PR.
5. Get the newly created PR link in your chat.

Using templates for PR descriptions

Bob automatically detects and uses your project's existing PR templates when generating pull request descriptions. This ensures your PRs are consistent with your team's established standards. When Bob detects multiple PR templates in your project, you can select which template to apply to your pull request.

Template detection

Bob searches for PR templates in the following locations relative to your project root:

1. `${cwd}/pull_request_template.md`
 2. `${cwd}/docs/pull_request_template.md`
 3. `${cwd}/.github/pull_request_template.md`
 4. `${cwd}/.github/PULL_REQUEST_TEMPLATE/pull_request_template.md`
-

5. `${cwd}/PULL_REQUEST_TEMPLATE/pull_request_template.md`

6. `${cwd}/docs/PULL_REQUEST_TEMPLATE/pull_request_template.md`

If Bob finds multiple templates, Bob will ask the user to choose which template to use for the PR description.

Default template

If no template is found in your project, Bob uses a built-in template that includes:

- A descriptive title derived from your branch name
- A summary of changes
- Key implementation details
- Testing information
- Any relevant links or references

Editing the generated description

After generating the PR description, you can choose to edit it before submission:

1. When prompted, select whether you want to edit the description.
2. If you choose to edit, Bob opens the description in a temporary markdown file.
3. Make your desired changes to the description.
4. Click the "Done" button when you have completed your edits.

If you choose not to edit, Bob uses the generated description as-is.

Customizing templates

For the best results with Bob's PR generation:

- Use clear section headings in your templates (e.g., `## Summary`, `## Changes`)
- Include placeholder text that provides context (e.g., `<!-- Describe the purpose of this PR -->`)
- Consider adding sections for testing instructions and related issues

Tips and best practices

- Use descriptive branch names to help Bob generate more accurate PR descriptions.
 - Review and edit the generated description before creating the PR.
-

Why use this feature?

- **Improved clarity:** Your prompts become more understandable for the AI model
- **Added context:** The enhancement process uses relevant context such as the current file path or selected code
- **Better instructions:** Your enhanced prompt contains specific guidance for the AI, such as formatting preferences or detail level.
- **Reduced ambiguity:** The enhancement process eliminates unclear language and ensures Bob understands your intent
- **Consistency:** Your prompts follow a standardized format that works well with the AI

How it works

1. **Type your initial prompt** in the Bob [chat interface](#) as you normally would.
- 2.

Click the
chat input box.

icon located in the bottom right of the

3. **Review the enhanced prompt** to make sure it accurately reflects your intent.
-

4. **Send the enhanced prompt** by pressing Enter or clicking the Send icon

()

Note:

The enhancement process works best with prompts that have a clear goal but need additional structure or context.

[Example transformations](#)

[Basic example](#)

Original prompt:

```
make a function to calculate tax
```

Enhanced prompt:

```
Create a JavaScript function that calculates sales tax. The function should:  
1. Take two parameters: price (number) and taxRate (decimal)  
2. Return the calculated tax amount  
3. Include proper error handling for invalid inputs  
4. Include comments explaining the calculation logic  
  
Please provide the complete function implementation with an example of how to use it.
```

Limitations and best practices

- **Experimental feature:** Prompt enhancement is still experimental. The quality of enhanced prompts may vary depending on your request's complexity and the model's capabilities.
- **Review carefully:** Always review the enhanced prompt before sending it. The enhancement process might make changes that do not align with your intentions.
- **Iterative refinement:** You can apply Enhance Prompt multiple times to iteratively refine your prompt until it is exactly what you want.
- **Start with clear intentions:** While enhancement improves your prompts, starting with a clear idea of what you want will produce the best results.
- **Context matters:** Enhancement works best when Bob has context about your current task. If you are starting a new conversation, provide some initial context before using Enhance Prompt.

By using Enhance Prompt, you can get more accurate and helpful responses from Bob while saving time on writing detailed prompts from scratch.

Why use literate coding?

- **Stay in your workflow:** Write code without context switching to chat interfaces
- **Maintain precise control:** Specify exactly where and how AI should modify your code
- **Express intent naturally:** Describe what you want in plain language or pseudocode
- **See changes before committing:** Review diffs to ensure the generated code meets your needs

How it works

When you activate literate coding mode, you can write natural language instructions, pseudocode, annotations, or partial code snippets directly in your source file. These instructions appear highlighted in blue to distinguish them from regular code.

When you run the Generate command, Bob:

1. Analyzes the entire file including your literate coding blocks
 2. Understands the surrounding context and your intent
 3. Replaces your instructions with real, executable code
 4. Shows you an inline diff so you can review the changes
-

This approach combines the expressiveness of natural language with the precision of direct code editing.

Getting started

Basic usage

1. Toggle literate coding mode:
 - Press `Cmd+I` (Mac) or `Ctrl+I` (Windows/Linux)
 - Or click the magic wand icon in the editor toolbar
2. Write your instructions:
 - Type pseudocode, natural language instructions, or annotations
 - Your text appears in blue, marking it as literate coding content
3. Generate code:
 - Press `Cmd+Enter` (recommended) or click the **Generate** button
 - Bob converts your instructions into real code and shows an inline diff
4. Review and apply:
 - **Accept** changes: `Cmd+Enter`
 - **Reject** changes: `Cmd+Shift+Backspace`
 - Changes are written to your file before acceptance, allowing you to test before deciding

Learn how to [generate code from comments](#) using literate coding.

Tips and best practices

- Be specific about your requirements in your instructions
- Include relevant context like variable types or function parameters
- For complex logic, break down your instructions into smaller, focused blocks
- Use iterate coding for both creating new code and modifying existing code

Current limitations

- **Single-file scope:** Currently works on one file at a time (multi-file support planned)
- **Preview feature:** Expect ongoing improvements to styling, diffing, and AI behavior

Skills are reusable instruction sets that teach Bob new workflows and specialized tasks. Think of them as recipes that Bob follows to complete specific types of work in a consistent, repeatable manner.

Why use skills

- **Reusability:** Define workflows once and use them across multiple conversations.
- **Consistency:** Ensure Bob follows the same approach every time for specific tasks.
- **Specialization:** Create domain-specific expertise for code reviews, testing, documentation, and more.
- **Team collaboration:** Share standardized workflows across your team through version control.
- **Flexibility:** Include supporting files like checklists, templates, and reference materials.

Requirements

Skills are only available in Advanced mode. This ensures Bob has access to all necessary tools to run skill-based workflows effectively.

How skills work

When you activate a skill, Bob receives the skill's instructions and gains access to any supporting files in the skill directory. Bob then follows these instructions to complete your task according to the defined workflow.

Skills load once per conversation to avoid duplicate prompts. Bob automatically determines when to activate a skill based on your request and the skill's description.

Creating a skill

Basic setup

Create a folder inside `.bob/skills/` in your project root, or use `~/ .bob/skills/` for global skills.

Add a `SKILL.md` file inside that folder.

Example structure:

SKILL.md

[SKILL.md format](#)

The `SKILL.md` file uses YAML front matter followed by the skill instructions:

```
---
name: code-review
description: Review code for bugs, security issues, and best practices
---

When reviewing code, check for:
- Security vulnerabilities
- Performance issues
- Missing error handling at API boundaries
- Unused imports and dead code

Provide a summary with severity levels for each finding.
```

Required fields:

- `name`: The skill's display name used in the Bob interface
- `description`: A clear summary that helps Bob decide when to activate this skill—skills without descriptions are ignored

Instructions section: Everything below the `---` delimiter becomes the instructions Bob receives when the skill is activated.

Adding supporting files

You can include additional files and subfolders alongside `SKILL.md` to provide reference materials, templates, checklists, scripts, or other resources. Bob can read these files automatically once the skill is activated.

Example with supporting files:

SKILL.md

checklist.md

severity-guide.md

analyze.sh

report-generator.py

Supporting files and folders can include:

- Checklists and templates
- Reference documentation
- Configuration examples
- Scripts and utilities in subfolders
- Style guides
- Any other resources needed for the skill

Skill locations

Skills can be defined at two levels:

Location	Scope	Use case
<project>/bob/skills/	Project-specific	Workflows unique to this project
~/bob/skills/	Global	Personal or organization-wide workflows

Priority: If both locations contain a skill with the same name, the project-level skill takes precedence.

Approving skills

By default, Bob asks for your permission before activating a skill. This gives you control over when specialized workflows are applied.

To skip the approval prompt:

Open Bob Settings.

Navigate to the Auto-Approve section.

Turn on the toggle for **Skills**.

With this setting enabled, Bob automatically activates skills when appropriate without requesting permission.

Writing effective skills

Clear descriptions

Write descriptions that clearly indicate when the skill should be used. Bob relies on these descriptions to determine skill relevance.

Good: "Review code for bugs, security issues, and best practices"

Avoid: "Code review skill"

Focused instructions

Keep the main `SKILL.md` file focused on the core workflow. Move detailed reference material, examples, and checklists to supporting files.

Example:

```
---
name: api-documentation
description: Generate API documentation following OpenAPI standards
---

Generate API documentation that includes:
- Endpoint descriptions
- Request/response schemas
- Authentication requirements
- Example requests and responses

Follow the style guide in `api-style-guide.md` and use the template in `api-
template.md`.
```

Actionable steps

Structure instructions as clear, actionable steps that Bob can follow systematically.

Example:

```
---
name: feature-implementation
description: Implement new features following team standards
---

<Steps>
<Step>
Review the feature requirements.

</Step>

<Step>
Create a plan with file changes needed.

</Step>

<Step>
Implement the feature with tests.

</Step>

<Step>
Update documentation.

</Step>

<Step>
Verify all tests pass.

</Step>
</Steps>

Follow coding standards in `coding-standards.md`.
```

Example skills

Code review skill

```

---
name: security-review
description: Review code for security vulnerabilities and best practices
---

Perform a security-focused code review:

<Steps>
<Step>
Check for common vulnerabilities:
- SQL injection risks
- XSS vulnerabilities
- Authentication/authorization issues
- Sensitive data exposure

</Step>

<Step>
Review security best practices:
- Input validation
- Output encoding
- Secure configuration
- Error handling

</Step>

<Step>
Provide findings with:
- Severity level (Critical, High, Medium, Low)
- Location in code
- Recommended fix
- Reference to security standards

</Step>
</Steps>

Use the severity guide in `severity-levels.md` for classification.

```

Documentation skill


```

---
name: api-docs
description: Generate comprehensive API documentation
---

Create API documentation that includes:

<Steps>
<Step>
Overview section:
- Purpose and use cases
- Authentication requirements
- Base URL and versioning

</Step>

<Step>
For each endpoint:
- HTTP method and path
- Description and purpose
- Request parameters
- Request body schema
- Response codes and schemas
- Example requests and responses

</Step>

<Step>
Additional sections:
- Error handling
- Rate limiting
- Pagination

</Step>
</Steps>

Follow the template in `api-doc-template.md` and examples in `api-examples.md`.

```

Tips and best practices

- **Start simple:** Begin with basic instructions and refine based on results.

- **Use supporting files:** Keep `SKILL.md` concise by moving detailed content to companion files.
- **Test thoroughly:** Verify skills work as expected before sharing with your team.
- **Version control:** Include project skills in your repository for team consistency.
- **Clear naming:** Use descriptive skill names that indicate their purpose.
- **Single responsibility:** Create focused skills for specific tasks rather than trying to handle multiple unrelated workflows.

Accessing Bobalytics

Note:

Bobalytics is available exclusively for Enterprise plan users.

Open Bobalytics from the IBM Bob web portal.

Go to bob.ibm.com and log in.

Click **Admin**, then select the **Bobalytics** tab.

Understanding Bob's impact

Track how much of Bob's contributions make it into your codebase to measure real-world impact on your software delivery lifecycle.

Bob Factor

The Bob Factor is the metric for understanding Bob's impact. It tracks the percentage of code, documentation, and plans that Bob created that were committed to your code repositories.

Bob's contribution by repository

This view shows which code repositories benefit most from Bob's assistance. Use it to:

- Identify repositories where Bob has the highest impact.
- Compare Bob's contribution across different projects.
- Understand adoption patterns across your codebase.

Tracking Bobcoin spending

Monitor your Bobcoin consumption to manage costs and optimize usage patterns.

Total spend

View your total Bobcoin expenditure for the selected time period. Compare current spending with the previous period to identify trends.

The "previous period" depends on your selected timeframe. By default, the timeframe is the last 4 weeks, and the previous period is the 4 weeks before that. If you change the timeframe to 2 weeks, the previous period becomes the 2 weeks before that.

Spend over time

Track how your Bobcoin consumption changes over time. Use this chart to:

- Identify spending spikes and investigate their causes
- Plan your Bobcoin budget based on historical trends
- Compare spending patterns across different time periods

Tracking user adoption

Monitor how your team adopts and uses IBM Bob across your organization.

Team activity

View overall team engagement with Bob. This metric helps you:

- Understand adoption rates across your organization
- Identify teams or individuals who might benefit from additional training
- Track growth in Bob usage over time

Daily active users

Track the number of unique users engaging with Bob each day. Use this metric to:

- Monitor consistent usage patterns

- Identify trends in daily engagement
- Measure the impact of training or onboarding initiatives

Bob's contribution by repository

In addition to showing Bob's impact, this view also helps track adoption by revealing which repositories actively use Bob's assistance.

Analyzing usage patterns

Understand how you and your team use Bob to identify opportunities for optimization and training.

Programming languages

See which programming languages Bob helps with most frequently. This data helps you:

- Understand where Bob provides the most value

- Identify languages where additional training might help
- Compare language usage across team members

Bob modes

Review which Bob modes you and your team use most often. Use this information to:

- Identify the most valuable modes for your workflows
- Discover underutilized modes that might improve productivity
- Share best practices for mode usage across your team

Individual versus team usage

Bobalytics provides two views for analyzing data:

Individual usage

Review your personal activity and metrics:

- Your Bobcoin consumption
- Your mode usage patterns
- Your activity timeline
- Your language breakdown
- Your commit activity with Bob's assistance

Team usage

View the same analytics aggregated across your organization. Team usage data is available to all Enterprise plan users and does not require administrator access.

Improving your workflow

Use Bobalytics insights to optimize how you work with Bob:

- Review which activities consume the most Bobcoins
 - Compare your usage patterns with team averages
 - Identify opportunities to increase Bob's impact on your codebase
-

- Set practical usage goals based on historical trends
- Share successful patterns with your team

Bobcoins are resource units that measure your consumption of Bob's AI capabilities. Each action you complete with Bob, such as generating code, analyzing files, or running commands, consumes a number of Bobcoins based on the computational resources required.

Base allocation

Your plan determines your allocation of Bobcoins.

Each individual plan includes a monthly allocation of Bobcoins. The following table states the amount of Bobcoins included with each plan:

Plan	Monthly Bobcoins included
Free trial	40 Bobcoins
Pro	40 Bobcoins
Pro+	160 Bobcoins
Ultra	500 Bobcoins

- Your Bobcoin allocation resets at the start of each billing cycle.
- 1 Bobcoin = \$0.50 USD.
- Pay-as-you-go overage system: You only pay for what you use beyond your base allocation.

Users with an Enterprise plan can purchase Bobcoin packs with the following details:

- **Amount:** 1,000 Bobcoins per pack.
- **Cost:** \$500 USD per pack.
- **Quantity:** Purchase as many packs as you need.

Warning

Bobcoins from packs expire 1 year after the purchase date.

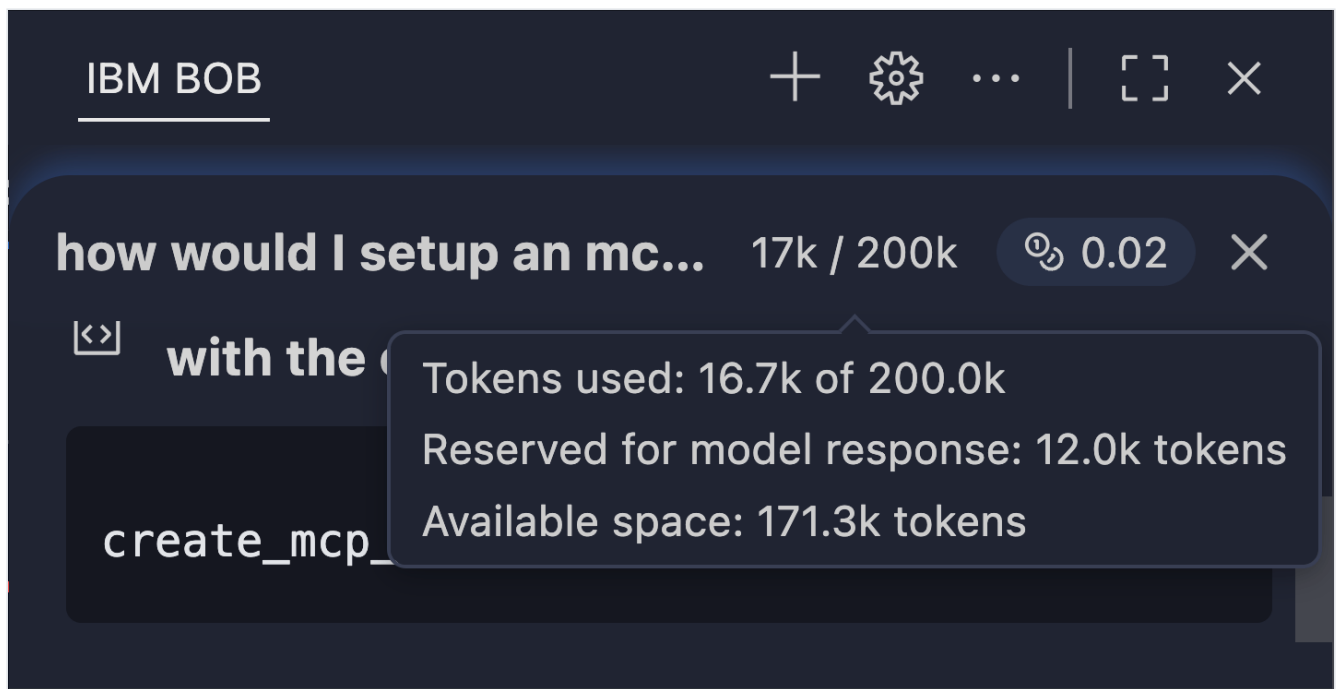
To purchase Bobcoin packs for your Enterprise plan, contact your IBM Sales representative.

[Bobcoins vs. tokens](#)

Understanding the relationship between tokens and Bobcoins helps clarify why IBM uses Bobcoins as the billing metric instead of raw token counts.

While Bobcoins measure your billing usage, the underlying AI models consume tokens, which are the fundamental units of text processing used by language models.

- **Tokens:** The raw units of model usage. Tokens represent pieces of text, such as words and characters, that the AI model processes. Every request to the model consumes input tokens, such as your prompt and context, and output tokens, which are Bob's responses.
- **Bobcoins:** IBM's billing metric that abstracts token usage into a predictable cost structure. One Bobcoin represents a standardized amount of computational resources, which includes token usage across different models and operations.



Since different AI models have varying token costs and processing requirements, Bobcoins provide a unified billing metric that simplifies cost prediction across all Bob operations, regardless of which underlying model is used.

You do not need to track tokens directly, because Bob automatically converts your model usage into Bobcoins for transparent billing. Different models and operations might consume tokens at different rates, but Bobcoins give you a consistent way to understand and predict your costs.

Overage

You can optionally enable an overage system to pay for more Bobcoins as you use them beyond your monthly allocation.

Note:

Overage costs do not apply to free trial or enterprise plans.

Enabling overage

Warning:

Once overage is enabled, you cannot disable it until the month ends.

To enable overage:

Navigate to the **Subscription** tab on the [Bob Administration](#) page.

Click **Edit**.

Select or enter the overage limit you want to set. Click **Save**.

The value you configure represents the overage limit measured in Bobcoins. Your total available spending capacity equals your plan limit plus your overage limit.

[Overage behavior and restrictions](#)

Once overage is enabled for the current billing month, the following restrictions apply:

- **Monthly reset:** The overage toggle automatically resets to OFF at the start of each new month.
- **Manual re-enablement:** You must manually re-enable overage each month if you want to continue using overage.
- **Adjustable limits:** After enabling overage, you can adjust the Bobcoin overage limit by either increasing or decreasing the amount at any time during the month.

[Overage billing](#)

- **Rate:** \$0.50 USD per Bobcoin.
- **Automatic billing:** Overages are automatically added to your monthly bill.

[Monitoring your usage](#)

You can track your Bobcoin usage and remaining budget in several ways:

- **Gauge component:** Hover your cursor over the gauge component located in the top-right corner of the Bob panel to see your current usage.
- **Settings page:** View detailed account information and usage statistics in Settings → General tab.
- **Bobalytics:** Use [Bobalytics](#) to track your usage patterns over time.

Ownership and administration

API keys are scoped to both the user and the subscription instance. Individual users can create, list, and revoke their own API keys. Subscription instance admins can list and revoke API keys for all users in their instance.

Subscription

Users

Teams

API Keys

API Keys

Total API Keys71

Active20

Expired1

Revoked50

Search API keys...

All

Create +

Name	Created By	Scope	Date created	Status	
Production Inference Key	john@acme.com	Inference	Apr 22, 2026, 1:29 AM	Active	
Development Testing Key	arvind@acme.com	General	Apr 16, 2026, 9:38 PM	Active	
API Key	rob@acme.com	Inference	Apr 16, 2026, 12:03 PM	Active	
API Key	neel@acme.com	General	Apr 16, 2026, 12:02 PM	Active	
Sarah's Personal Key	bob@acme.com	General	Apr 16, 2026, 10:51 AM	Revoked	
API Key	bob@acme.com	General	Apr 16, 2026, 10:47 AM	Active	
CI/CD Pipeline Key	bob@acme.com	Inference	Apr 16, 2026, 2:33 AM	Active	
Michael's Development Key	bob@acme.com	General	Apr 16, 2026, 2:32 AM	Active	
Emma's Testing Key	bob@acme.com	General	Apr 16, 2026, 2:24 AM	Active	

[API key types](#)

You can create two kinds of API keys:

Type	Scope and usage
General	Scoped to your subscription instance and usable across multiple services. For inference requests, you must also include the team ID header.
Inference	Scoped to a specific subscription instance and team, and limited to inference requests only. No additional instance or team headers are required.

[When to use API keys](#)

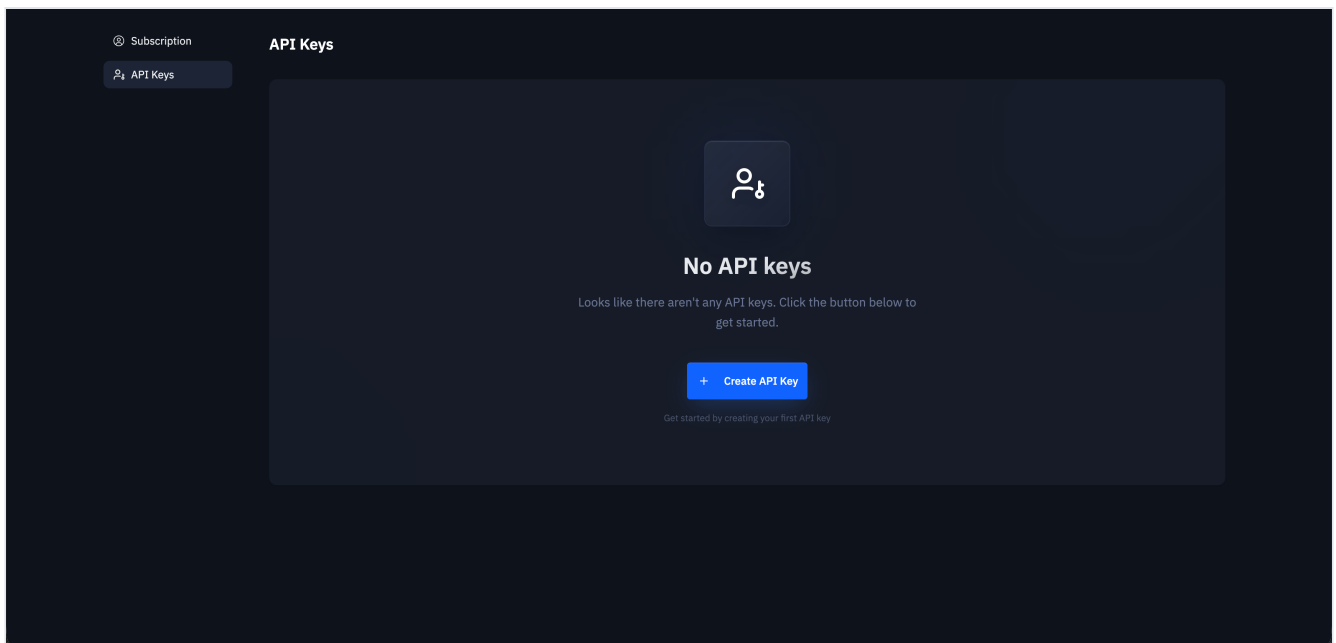
Use API keys when you need to:

- Authenticate Bob Shell in non-interactive sessions
- Run Bob in CI/CD pipelines or scheduled automation
- Avoid browser-based sign-in for scripted workflows
- Authenticate inference requests with either a general or inference API key

Create an API key

Go to bob.ibm.com and log in.

Open your subscription instance and navigate to the API key management section.



Create a new API key and choose the key type you want to create.

Create API key

Name

API Key

Description (optional)

Description for API key

Scope

General

Inference

ⓘ

Scoped to your instance and usable across multiple services; however, for inference requests, you must include the team ID header.

Expiration date (optional)

📅

Expiration date

Create

Copy the API key value and store it securely.

Warning:

You will not be able to view the API key value again after creation.

Manage your API keys

You can use the API key management section to:

- List your API keys
- Search and filter keys
- View API key details
- Revoke active keys
- Show revoked keys alongside active and expired keys

Administrator capabilities

Subscription instance admins can manage API keys across their instance. Admin capabilities include:

- Listing API keys for all users in the instance
- Revoking API keys for any user in the instance
- Viewing active, revoked, and expired keys

Use an API key with Bob Shell

To use an API key with Bob Shell, see [Install and set up Bob Shell](#) and [Starting a non-interactive session](#).

Use these guidelines to reduce risk when you work with Bob in your development environment.

Security checklist

Review this checklist before you use Bob in a new workspace:

- Configure `.bobignore` to restrict file access.
- Review and limit auto-approve settings.
- Keep secrets out of prompts, code snippets, and accessible files.
- Use Model Context Protocol (MCP) servers with authentication, encryption, and access controls.
- Review Bob's output before you apply changes or run generated commands.

Restrict file access with `.bobignore`

You can limit which files Bob reads or modifies by configuring `.bobignore`. The `.bobignore` file uses the same syntax as `.gitignore` and is one of the first controls to configure in a workspace.

[Set up .bobignore](#)

Create a `.bobignore` file in your workspace root.

Add patterns for sensitive files, credential stores, generated assets, and any data that Bob must not access.

```
# Example .bobignore patterns
.env
secrets/
*.key
config/credentials.json
```

Bob monitors the `.bobignore` file and applies changes automatically. For more information, see [Using .bobignore](#).

[Understand .bobignore limitations](#)

`.bobignore` helps control access through Bob tools, but it does not isolate Bob from your system. Keep these limitations in mind:

- It applies only to files in the current workspace.
- It does not create a system-level sandbox.
- You still need to review tool permissions and approval settings carefully.

Limit auto-approve settings

You can configure Bob to skip confirmation prompts for some actions. Auto-approve can speed up routine work, but it also increases risk.

Warning

Auto-approve settings reduce your opportunity to review actions before Bob reads files, edits content, runs commands, or uses external tools. Use auto-approve only after you understand the risk for your workspace.

Review high-risk settings

Setting	Risk	Recommendation
Edit files	High	Enable only in controlled environments.
Execute commands	High	Restrict usage and avoid broad command patterns.
Use MCP servers	Medium to high	Use only trusted servers with clear access controls.

Setting	Risk	Recommendation
Read files	Medium	Consider whether accessible files contain sensitive data.

Always review Bob output for accuracy before you apply changes or run generated commands. Do not assume AI-generated output is correct or safe.

For more information, see [Auto-approving actions](#).

Handle secrets securely

Do not provide secrets directly to Bob or any other AI system. Even short-lived exposure in prompts, pasted code, or accessible files can create risk.

Follow secrets management practices

- Store secrets in environment files or approved secret management systems.
- Add secret files to both `.gitignore` and `.bobignore`.
- Use secret management tools when possible.
- Apply the principle of least privilege when you assign credentials.

Use delegated access when needed

Do not use credentials that let AI systems act broadly on your behalf without review. When Bob needs access to external systems:

- Use delegated access methods such as OAuth when available.
- Use time-limited tokens.
- Monitor and audit all actions.

Use MCP securely

Model Context Protocol (MCP) extends Bob by connecting it to external tools and services. These connections can increase capability, but they also expand your security boundary.

Understand the MCP security model

In an MCP integration:

- Bob acts as the host with an MCP client.
- The client connects to local or remote MCP servers.
- MCP servers expose additional tools and actions.

Secure MCP servers

When you use MCP servers, verify these controls:

- **Authentication:** Verify the identity of users and services that access the server.
- **Encryption:** Protect data in transit between Bob and the server.
- **Access controls:** Limit which actions the server can perform.
- **Auditing:** Record actions so you can investigate activity later.

Treat remote MCP servers like other production or shared infrastructure. Apply endpoint protection, network restrictions, and access controls that match the sensitivity of the systems they reach.

For shared MCP servers, make sure that actions are attributable to specific users or sessions.

Additional security practices

- Review generated content before you implement it.
- Keep Bob and related dependencies up to date.
- Train team members on AI security risks and review expectations.
- Prepare an incident response process for AI-assisted workflows.
- Use dedicated workspaces for sensitive projects when appropriate.
- Scope permissions to the minimum access required for each task.

These practices help you use Bob more safely in everyday development workflows.

Shared Bobcoin pool

Distribute Bobcoins from a centralized budget to empower your entire team.

Admin dashboard

Control spending, manage users, and track usage across your organization.

Enterprise support

Get assistance from dedicated IBM Bob specialists.

Scale on demand

Purchase additional Bobcoin packs as your team grows.

Enterprise features

Bobalytics

Track Bob's impact on your software delivery lifecycle with comprehensive analytics and insights. Bobalytics helps you:

- Measure Bob's contribution to your codebase.
- Monitor Bobcoin spending across your organization.
- Analyze usage patterns by programming language and mode.

To learn more, see [Bobalytics](#).

Centralized management

Take control of your organization's AI development with a dedicated administrative dashboard:

- **Shared Bobcoin pool:** Distribute Bobcoins to team members.
- **User management:** Use Seats to add, remove, and manage team member access.
- **Spending controls:** Set individual spending limits and adjust based on your team needs.
- **Usage insights:** Monitor Bobcoin consumption and optimize your resource allocation.

Flexible scaling

Enterprise plans grow with your organization. Purchase additional Bobcoin packs when your team needs more capacity, and distribute resources based on individual or team requirements.

Support

Enterprise customers get access to dedicated assistance from IBM Bob specialists. If you already have an enterprise plan, you can open a support case on the [IBM Support](#) website.

[Getting started](#)

Your IBM Bob Enterprise plan provides access to the [Administration](#) page, where you can administer your team and configure enterprise features.

[Managing your subscription](#)

To manage your IBM Bob Enterprise plan, including upgrades, downgrades, or billing changes, contact your IBM Sales representative.

Ready to bring IBM Bob to your organization? Contact our team to discuss your needs and get a customized solution.

[View pricing](#)

[Explore Enterprise plan options and pricing details.](#)

[Contact sales](#)

[Speak with our team about your organization's needs.](#)

Connection and authentication

Configure your network firewall to allow Bob connectivity.

For more information, see [Configuring firewall rules](#).

Installing and updating

Resolve performance issues related to incorrect version installation.

For more information, see [Slow performance after installation or update](#).

Interface and layout

Reposition the Bob panel to your preferred sidebar location.

For more information, see [Repositioning the Bob panel](#).

When all else fails

Reset Bob when standard troubleshooting steps fail.

For more information, see [Resetting Bob](#).